

Airflow

What is the Local Executor?

- Run multiple tasks in parallel
- Based on the Python module: multiprocessing
- Spawn processes to execute tasks
- Easy to set up
- Resource light
- Vertical Scaling
- Single point of failure
- As a best practice, always try the Local Executor first! KISS (Keep it simple, stupid)

Parallel Execution enables simultaneous execution of tasks, which enhances performance and reduces the time required for task completion.

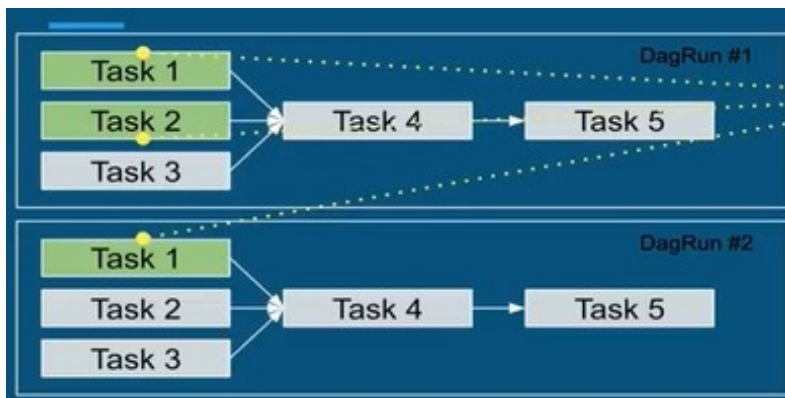
What is PostgreSQL?

PostgreSQL is an object-relational database management system that executes using SQL, being used in t

- Object-relational database
- ACID compliant
- Client-server database
- Handle concurrent connections
- SQL standard and advanced
- Highly scalable
- Extensible

There's a parameter for concurrency and parallelism: If parallelism=3, 3 tasks can be run simultaneously FOR ALL DAGRuns.

If max_active_runs_per_dag = 1, then only DAGRun #1 will run, once finished, then DAGRun #2 is run, just as following in the image below:



If dag_concurrency=2 up to 2 tasks can be run PER DAGRun. In the DagRun #1, the third task can't be run simultaneously since dag_concurrency=2.

Local Executor strategies

- parallelism = 0
 - "Unlimited" parallelism
- parallelism > 0
 - Limited parallelism

```
airflow-materials > airflow-section-5 > docker-compose-LocalExecutor.yml
1  version: '2.1'
2  services:
3    postgres:
4      image: postgres:9.6
5      environment:
6        - POSTGRES_USER=airflow
7        - POSTGRES_PASSWORD=airflow
8        - POSTGRES_DB=airflow
9    webserver:
10     build: ./docker/airflow
11     restart: always
12     depends_on:
13       - postgres
14     environment:
15       - LOAD_EX=n
16       - EXECUTOR=Local
17     volumes:
18       - ./mnt/airflow/dags:/usr/local/airflow/dags
19       - ./mnt/airflow/airflow.cfg:/usr/local/airflow/airflow.cfg
20     ports:
21       - "8080:8080"
22     command: webserver
23     healthcheck:
24       test: ["CMD-SHELL", "[ -f /usr/local/airflow/airflow-webserver.pid ]"]
25       interval: 30s
26       timeout: 30s
27       retries: 3
28
```

This makes so only one task can be run in parallel.

```
airflow-materials > airflow-section-5 > mnt > airflow > airflow.cfg
98  # The amount of parallelism as a setting to the executor. This defines
99  # the max number of task instances that should run simultaneously
100 # on this airflow installation
101 parallelism = 1
```

```

Directory of C:\Users\pedro\Lamia\Bootcamp\Tarefa 18\P2\airflow-materials\airflow-section-5
11/21/2024 10:18 PM <DIR> .
11/21/2024 10:18 PM <DIR> ..
11/21/2024 10:18 PM 6,148 .DS_Store
11/21/2024 10:18 PM <DIR> .vagrant
11/21/2024 10:18 PM <DIR> dag_solutions
11/21/2024 10:18 PM <DIR> docker
11/21/2024 10:18 PM 3,037 docker-compose-CeleryExecutor.yml
11/21/2024 10:18 PM 777 docker-compose-LocalExecutor.yml
11/21/2024 10:18 PM <DIR> docs
11/21/2024 10:18 PM <DIR> mnt
11/21/2024 10:18 PM <DIR> vagrant-files
3 File(s) 9,962 bytes
8 Dir(s) 206,326,390,784 bytes free

C:\Users\pedro\Lamia\Bootcamp\Tarefa 18\P2\airflow-materials\airflow-section-5>docker-compose -f docker-compose-LocalExe
cutor.yml up -d

```

Airflow Metadata Database and Local Executor Configuration

I explored how to interact with a Postgres database, query metadata, and manage the Airflow environment. Additionally, security considerations were emphasized, particularly regarding the configuration of the `sql_mode` parameter in the `airflow.cfg` file. Best practices for securing the database while using Airflow were discussed, with a recommendation to keep Airflow focused on orchestration rather than database exploration.

Starting the Web Server:

To start the web server, the following command is used:

```
docker-compose -f docker-compose-LocalExecutor.yml up -d
```

In this case, the `docker-compose` function is basing its process in the `docker-compose-LocalExecutor.yml` file.

Docker and Postgres Configuration Setup

I began by setting up the local executor with Postgres using a Docker Compose configuration. The `docker-compose-LocalExecutor.yml` file defines two services:

- **Postgres:** The database is configured using the Postgres 9.6 image, with environment variables for the user, password, and database name.
- **Webserver:** This service depends on Postgres and includes the Airflow configuration, such as the local executor setup, port exposure, and health checks. It's important to note that both the scheduler and web server run in the same container in this configuration.

Getting data

Now, airflow can be opened in `localhost:8080`. A task is set to run just so it can generate some data to explore:

Airflow DAGs interface showing a list of DAGs and a dropdown menu for Data Profiling.

The top navigation bar includes: Airflow, DAGs, Data Profiling, Browse, Admin, Docs, About. The date and time are 2020-01-20 17:47:06 UTC.

DAGs

Search:

	DAG	Schedule	Owner	Recent Tasks	Last Run	DAG Runs	Links
☒	next_dag	00***	Airflow				
☒	parallel_dag	00***	Airflow		2019-01-04 00:00		
☒	pool_dag	00***	Airflow				
☒	queue_dag	00***	Airflow				

Showing 1 to 4 of 4 entries

Hide Paused DAGs

The Data Profiling dropdown menu is open, showing options: Ad Hoc Query, Charts, Known Events.

But first, a connection must be created:

Connection [create]

List Create

Conn Id * postgres_1

Conn Type Postgres

Host postgres

Schema airflow

Login airflow

Password *****

Port 5432

Extra

Save Save and Add Another Save and Continue Editing Cancel

Note: all the configuration put in that connection was based from the entrypton.sh file.

After it, SQL commands can be used into Ad Hoc Query:

Ad Hoc Query

postgres_1 **Run** .csv

1 SELECT * FROM dag_run;

Show 100 entries

task_id	dag_id	execution_date	start_date	end_date	duration	state	try_number	hostname	username	job_id
task_5	parallel_dag	2019-01-03 00:00:00+00:00	2020-01-20 17:46:54.721756+00:00	2020-01-20 17:46:56.010858+00:00	1.2891	success	1	5df342cb939d	airflow	16
task_4	parallel_dag	2019-01-04 00:00:00+00:00					0		airflow	
task_5	parallel_dag	2019-01-04 00:00:00+00:00					0		airflow	
task_2	parallel_dag	2019-01-01 00:00:00+00:00	2020-01-20 17:45:32.822310+00:00	2020-01-20 17:45:39.411415+00:00	6.5891	success	1	5df342cb939d	airflow	3
task_3	parallel_dag	2019-01-01 00:00:00+00:00	2020-01-20 17:45:32.799138+00:00	2020-01-20 17:45:39.412613+00:00	6.61348	success	1	5df342cb939d	airflow	2
task_1	parallel_dag	2019-01-01 00:00:00+00:00	2020-01-20 17:45:42.681942+00:00	2020-01-20 17:45:48.974736+00:00	6.29279	success	1	5df342cb939d	airflow	4
task_2	parallel_dag	2019-01-04 00:00:00+00:00	2020-01-20 17:47:02.784482+00:00	2020-01-20 17:47:09.220757+00:00	6.43628	success	1	5df342cb939d	airflow	17
task_1	parallel_dag	2019-01-04 00:00:00+00:00	2020-01-20 17:47:02.811862+00:00	2020-01-20 17:47:09.281788+00:00	6.46993	success	1	5df342cb939d	airflow	18
task_4	parallel_dag	2019-01-01 00:00:00+00:00	2020-01-20 17:45:52.683839+00:00	2020-01-20 17:45:54.151024+00:00	1.46718	success	1	5df342cb939d	airflow	5
task_3	parallel_dag	2019-01-04 00:00:00+00:00	2020-01-20 17:47:10.929949+00:00	2020-01-20 17:47:17.370011+00:00	6.44006	success	1	5df342cb939d	airflow	19
task_5	parallel_dag	2019-01-01 00:00:00+00:00	2020-01-20 17:45:56.623529+00:00	2020-01-20 17:45:57.910798+00:00	1.28723	success	1	5df342cb939d	airflow	6
task_3	parallel_dag	2019-01-02 00:00:00+00:00	2020-01-20 17:46:02.777260+00:00	2020-01-20 17:46:09.131432+00:00	6.35417	success	1	5df342cb939d	airflow	7
task_2	parallel_dag	2019-01-02 00:00:00+00:00	2020-01-20 17:46:02.873961+00:00	2020-01-20 17:46:09.209268+00:00	6.33531	success	1	5df342cb939d	airflow	8
task_1	parallel_dag	2019-01-02 00:00:00+00:00	2020-01-20 17:46:10.643514+00:00	2020-01-20 17:46:16.908210+00:00	6.2647	success	1	5df342cb939d	airflow	9
task_4	parallel_dag	2019-01-02 00:00:00+00:00	2020-01-20 17:46:18.793033+00:00	2020-01-20 17:46:20.128151+00:00	1.33512	success	1	5df342cb939d	airflow	10
task_5	parallel_dag	2019-01-02 00:00:00+00:00	2020-01-20 17:46:22.868174+00:00	2020-01-20 17:46:24.135145+00:00	1.26697	success	1	5df342cb939d	airflow	11
task_2	parallel_dag	2019-01-03 00:00:00+00:00	2020-01-20 17:46:30.694315+00:00	2020-01-20 17:46:37.040470+00:00	6.34616	success	1	5df342cb939d	airflow	12
task_3	parallel_dag	2019-01-03 00:00:00+00:00	2020-01-20 17:46:30.704080+00:00	2020-01-20 17:46:37.040708+00:00	6.33663	success	1	5df342cb939d	airflow	13
task_1	parallel_dag	2019-01-03 00:00:00+00:00	2020-01-20 17:46:38.688360+00:00	2020-01-20 17:46:44.960291+00:00	6.27193	success	1	5df342cb939d	airflow	14
task_4	parallel_dag	2019-01-03 00:00:00+00:00	2020-01-20 17:46:48.758116+00:00	2020-01-20 17:46:50.095831+00:00	1.33771	success	1	5df342cb939d	airflow	15

Showing 1 to 20 of 20 entries

Now it's just a matter of creativity to enter the needed commands.

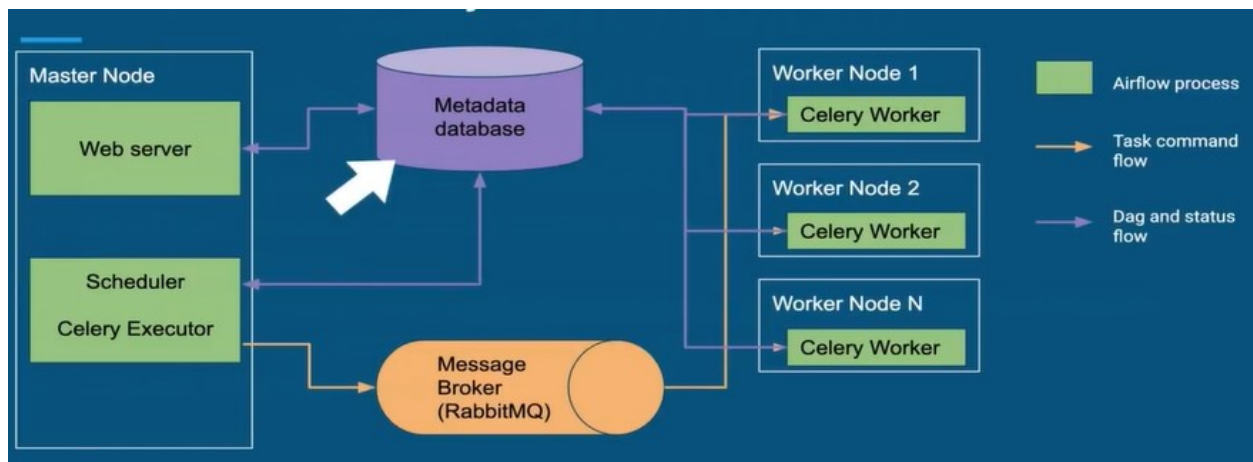
Scaling out Apache Airflow with Celery Executors

What is the Celery Executor?

- Scale out Apache Airflow
- Backed by Celery: Asynchronous Distributed Task Queue
- Distribute tasks among worker nodes
- `airflow worker`: Create a worker to execute tasks
- Horizontal Scaling
- High availability: If one worker goes down, Airflow can still schedule tasks
- Need a message broker: RabbitMQ/Redis
- Can set different queues

The Celery Executor is an excellent choice for scaling Apache Airflow in distributed environments. It enables horizontal scaling by distributing tasks across multiple worker nodes, ensuring efficient resource utilization. With high availability, Airflow can continue scheduling tasks even if individual workers fail, making it highly reliable. Additionally, it supports message brokers like RabbitMQ or Redis for communication and allows the configuration of different task queues, offering flexibility in workload management. This setup is ideal for handling complex workflows with high throughput and varying resource demands.

Here is a chart on how it works:



What are the drawbacks?

- Need an extra component: Message Broker Service
- Take time and work to set up
- Worker maintenance

In the file "docker-compose-CeleryExecutor.yml", it is possible to look at the different services:

airflow-materials > airflow-section-5 > 🚀 docker-compose-CeleryExecutor.yml

```
1  version: '2.1'
2  services:
3      redis:
4          image: 'redis:5.0.5'
5          command: redis-server --requirepass redispass
6
7      postgres:
8          image: postgres:9.6
9          environment:
10             - POSTGRES_USER=airflow
11             - POSTGRES_PASSWORD=airflow
12             - POSTGRES_DB=airflow
13             # Uncomment these lines to persist data on the local filesystem.
14             # - PGDATA=/var/lib/postgresql/data/pgdata
15             # volumes:
16             # - ./mnt/postgres:/var/lib/postgresql/data/pgdata
17
18      webserver:
19          build: ./docker/airflow
20          restart: always
21          depends_on:
22             - postgres
23             - redis
24          environment:
25             - LOAD_EX=n
26             - FERNET_KEY=46BKJoQYlPP0exq00hDZnIlNepKFf87WFwLbfzqDDho=
27             - EXECUTOR=Celery
28             - POSTGRES_USER=airflow
29             - POSTGRES_PASSWORD=airflow
30             - POSTGRES_DB=airflow
31             - REDIS_PASSWORD=redispass
32          volumes:
33             - ./mnt/airflow/dags:/usr/local/airflow/dags
34             - ./mnt/airflow/airflow.cfg:/usr/local/airflow/airflow.cfg
35             # Uncomment to include custom plugins
36             # - ./plugins:/usr/local/airflow/plugins
37          ports:
38             - "8080:8080"
39          command: webserver
40          healthcheck:
41             test: ["CMD-SHELL", "[ -f /usr/local/airflow/airflow-webserver.pid ]"]
42             interval: 30s
43             timeout: 30s
44             retries: 3
45
46      flower:
47          build: ./docker/airflow
48          restart: always
```

In the image, it's possible to see we're using redis as the message broker, for example.

In the image below, it shows the web server using a `FERNET_KEY`, which is used to encrypt the connections, will be better explained later on.

```
webserver:
  build: ./docker/airflow
  restart: always
  depends_on:
    - postgres
    - redis
  environment:
    - LOAD_EX=n
    - FERNET_KEY=46BKJoQYlPP0exq0hDZnIlNepKFf87WFwLbfzqDDho=
    - EXECUTOR=Celery
    - POSTGRES_USER=airflow
    - POSTGRES_PASSWORD=airflow
    - POSTGRES_DB=airflow
    - REDIS_PASSWORD=redispass
```

To compose using the file above:

```
docker-compose -f docker-compose-CeleryExecutor.yml up -d
```

```
Marc@MacBook-Pro ~/airflow-materials/airflow-section-5 (master) $ docker-compose -f docker-compose-CeleryExecutor.yml up -d
Creating network "airflow-section-5_default" with the default driver
Pulling redis (redis:5.0.5)...
5.0.5: Pulling from library/redis
b8f262c62ec6: Pull complete
93789b5343a5: Pull complete
49cdbb315637: Pull complete
2c1ff453e5c9: Pull complete
9341ee0a5d4a: Pull complete
770829e1df34: Pull complete
Digest: sha256:5dccb533dc0deacce4a02fe9035134576368452db0b4323b98a4b2ba2d3b302
Status: Downloaded newer image for redis:5.0.5
Creating airflow-section-5_redis_1 ... done
Creating airflow-section-5_postgres_1 ... done
Creating airflow-section-5_flower_1 ... done
Creating airflow-section-5_webserver_1 ... done
Creating airflow-section-5_scheduler_1 ... done
Creating airflow-section-5_worker_1 ... done
```

Besides the airflow server on the port 8080, it also opens the port for the flower server, on the port 5555:


```
webserver:
  build: ./docker/airflow
  restart: always
  depends_on:
    - postgres
    - redis
  environment:
    - LOAD_EX=n
    - FERNET_KEY=46BKJoQYlPPOexq00hDZnI
    - EXECUTOR=Celery
    - POSTGRES_USER=airflow
    - POSTGRES_PASSWORD=airflow
    - POSTGRES_DB=airflow
    - REDIS_PASSWORD=redispass
  volumes:
    - ./mnt/airflow/dags:/usr/local/airflow/dags
    - ./mnt/airflow/airflow.cfg:/usr/local/airflow/airflow.cfg
    # Uncomment to include custom plugins
    # - ./plugins:/usr/local/airflow/plugins
  ports:
    - "8080:8080"
  command: webserver
  healthcheck:
    test: ["CMD-SHELL", "[ -f /usr/local/airflow/airflow.cfg ]"]
    interval: 30s
    timeout: 30s
    retries: 3

flower:
  build: ./docker/airflow
  restart: always
  depends_on:
    - redis
  environment:
```

Here is the worker:

The screenshot shows the Flower dashboard at localhost:5555. The top navigation bar includes 'Dashboard', 'Tasks', 'Broker', and 'Monitor'. Below the navigation bar, there are summary statistics: Active: 0, Processed: 0, Failed: 0, Succeeded: 0, and Retried: 0. A search bar is located on the right. The main table lists workers with columns: Worker Name, Status, Active, Processed, Failed, Succeeded, Retried, and Load Average. One worker is listed: celery@631974cc909c, with status 'Online' and all other metrics at 0. The Load Average is shown as 0.71, 0.43, 0.19. A footer note says 'Showing 1 to 1 of 1 entries'.

Worker Name	Status	Active	Processed	Failed	Succeeded	Retried	Load Average
celery@631974cc909c	Online	0	0	0	0	0	0.71, 0.43, 0.19

Now entering airflow and starting a DAG:

The screenshot shows the Airflow DAGs page. It features a search bar and a table with columns: DAG, Schedule, Owner, Recent Tasks, Last Run, DAG Runs, and Links. Four DAGs are listed: next_dag, parallel_dag, pool_dag, and queue_dag. The 'parallel_dag' entry is highlighted with a red box. Below the table, there is a pagination control showing '1' and a note 'Showing 1 to 4 of 4 entries'. A link 'Hide Paused DAGs' is visible at the bottom left.

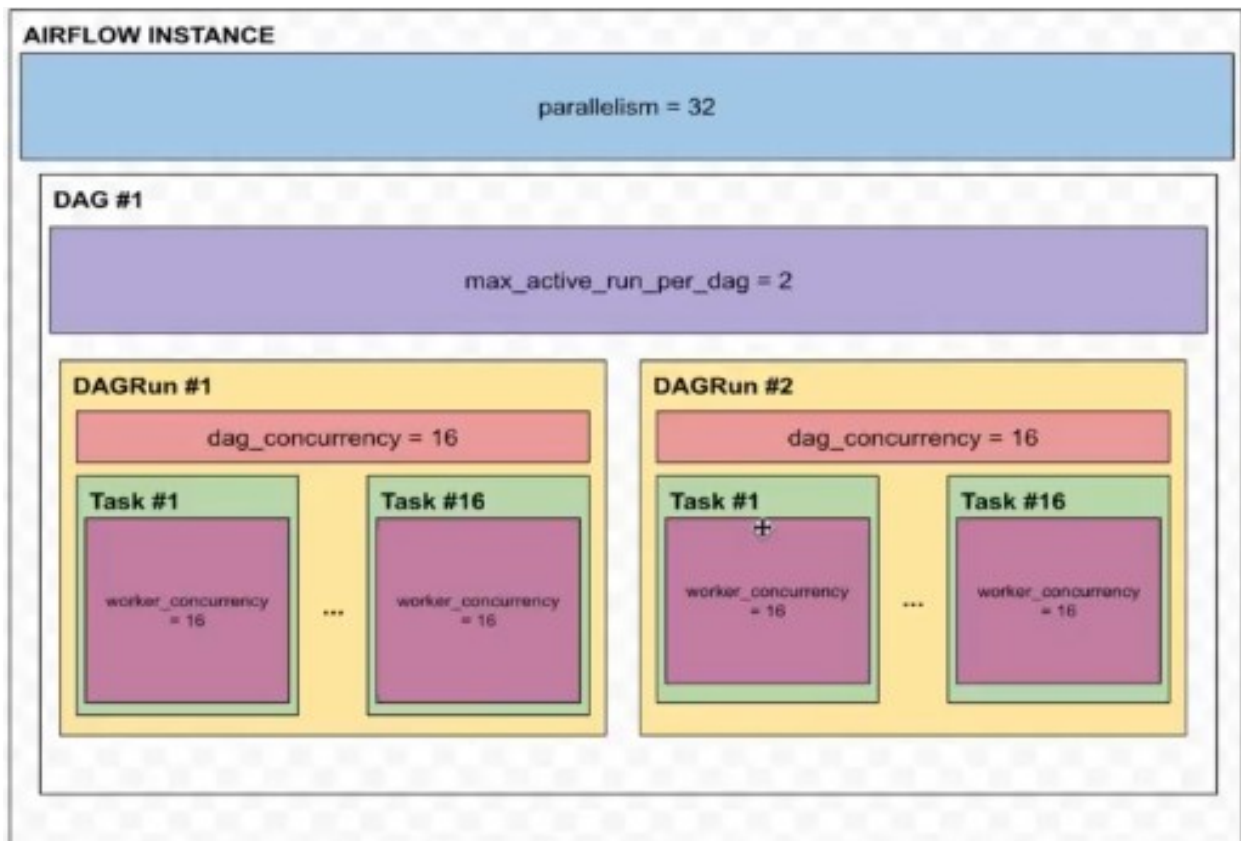
DAG	Schedule	Owner	Recent Tasks	Last Run	DAG Runs	Links
next_dag	00***	Airflow				
parallel_dag	00***	Airflow				
pool_dag	00***	Airflow				
queue_dag	00***	Airflow				

In the flower dashboard, it can be seen that the worker keeps track of the state of each task that is being executed in airflow, providing some interesting labels such as "Active", "Processed", "Failed", "Succeeded", "Retried" and "Load Average":

The screenshot shows the Flower dashboard with updated statistics: Active: 1, Processed: 18, Failed: 0, Succeeded: 17, and Retried: 0. The worker table now shows one worker: celery@da495f40f24f, with status 'Online', 1 Active task, 18 Processed tasks, 0 Failed tasks, 17 Succeeded tasks, 0 Retried tasks, and a Load Average of 0.95, 0.66, 0.57.

Worker Name	Status	Active	Processed	Failed	Succeeded	Retried	Load Average
celery@da495f40f24f	Online	1	18	0	17	0	0.95, 0.66, 0.57

"Load Average" keeps track of how intensely the CPU is being used to process all of the tasks, being useful for some insights in



How to increase the number of executed tasks to the worker node

To do it, it's needed to add new nodes to the cluster:

It's possible to do it manually, making it easy to set it regardless on which architecture is being worked on.

When using the docker-compose method, all the containers started are able to communicate with each other.

To do it manually, it's needed to expose the ports of the desired container:

```
Marc@MacBook-Pro ~/airflow-materials/airflow-section-5 (master) $ docker-compose up -d
Creating network "airflow-section-5_default" with the default driver
Creating airflow-section-5_postgres_1 ... done
Creating airflow-section-5_redis_1 ... done
Creating airflow-section-5_flower_1 ... done
Creating airflow-section-5_webserver_1 ... done
Creating airflow-section-5_scheduler_1 ... done
Creating airflow-section-5_worker_1 ... done
```

```
docker run --network airflow-section-5_default --expose 8793 -v
./mnt/airflow/dags:/usr/local/airflow/dags -v
./mnt/airflow/airflow.cfg:/usr/local/airflow/airflow.cfg python:3.7
```

Since the `-v` argument does not support relative paths, so wherever there's a `."` to generalize a path, it should be substituted by the actual path in the computer.

After it, in the `airflow.cfg` file, the parameters `"executor"` and `"sql_alchemy_conn"` should be changed for `"CeleryExecutor"` and `"postgresql+psycopg2://airflow:airflow@postgres:5432/airflow"`, respectively.

```
# The executor class that airflow should use. Choices include
# SequentialExecutor, LocalExecutor, CeleryExecutor, DaskExecutor, KubernetesExecu
executor = SequentialExecutor

# The SQLAlchemy connection string to the metadata database.
# SQLAlchemy supports many different database engine, more information
# their website
sql_alchemy_conn = sqlite:///usr/local/airflow/airflow.db
```

```
# The executor class that airflow should use. Choices include
# SequentialExecutor, LocalExecutor, CeleryExecutor, DaskExecutor, KubernetesExecu
executor = CeleryExecutor

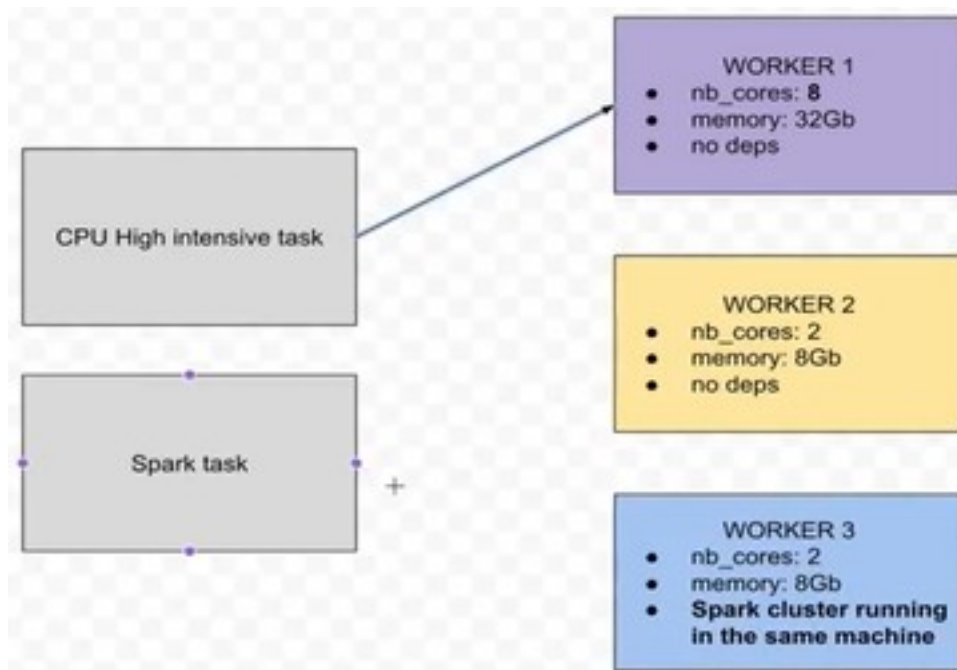
# The SQLAlchemy connection string to the metadata database.
# SQLAlchemy supports many different database engine, more information
# their website
sql_alchemy_conn = postgresql+psycopg2://airflow:airflow@postgres:5432/airflow
```

After it, the new worker node will be already working:

Worker Name	Status
celery@214852b5219b	Online
celery@961e45f1f05d	Online

Sending tasks to specific workers

It may be useful in the case of letting a specific worker with more dedicated ram/cpu to deal with a more intensive task, to optimize the system.



```
# Default queue that tasks get assigned to and that worker listen on.
default_queue = default
```

Creating 3 different queues, each one with an specific metric.

```
worker:
  build: ./docker/airflow
  restart: always
  depends_on:
    - scheduler
  volumes:
    - ./mnt/airflow/dags:/usr/local/airflow/dags
    - ./mnt/airflow/airflow.cfg:/usr/local/airflow/airflow.cfg
    # Uncomment to include custom plugins
    # - ./plugins:/usr/local/airflow/plugins
  environment:
    - FERNET_KEY=46BKJoQYlPP0exq00hDZnIlNepKFf87WFwLbfzqDDho=
    - EXECUTOR=Celery
    - POSTGRES_USER=airflow
    - POSTGRES_PASSWORD=airflow
    - POSTGRES_DB=airflow
    - REDIS_PASSWORD=redispass
  command: worker
```

```

worker:
  build: ./docker/airflow
  restart: always
  depends_on:
    - scheduler
  volumes:
    - ./mnt/airflow/dags:/usr/local/airflow/dags
    - ./mnt/airflow/airflow.cfg:/usr/local/airflow/airflow.cfg
    # Uncomment to include custom plugins
    # - ./plugins:/usr/local/airflow/plugins
  environment:
    - FERNET_KEY=46BKJoQYlPPOexq00hDZnIlNepKFf87WFwLbfzqDDho=
    - EXECUTOR=Celery
    - POSTGRES_USER=airflow
    - POSTGRES_PASSWORD=airflow
    - POSTGRES_DB=airflow
    - REDIS_PASSWORD=redispass
  command: worker -q worker_ssd, worker_cpu, worker_spark

```

Now in the terminal:

```
docker-compose -f docker-compose-CeleryExecutor.yml up -d --scale worker=3
```

```

Marc@MacBook-Pro ~/airflow-materials/airflow-section-5 (master) $ docker-compose -f docker-compose-CeleryExecutor.yml up -d --scale worker=3
Creating network "airflow-section-5_default" with the default driver
Creating airflow-section-5_postgres_1 ... done
Creating airflow-section-5_redis_1 ... done
Creating airflow-section-5_webserver_1 ... done
Creating airflow-section-5_flower_1 ... done
Creating airflow-section-5_scheduler_1 ... done
Creating airflow-section-5_worker_1 ... done
Creating airflow-section-5_worker_2 ... done
Creating airflow-section-5_worker_3 ... done
Marc@MacBook-Pro ~/airflow-materials/airflow-section-5 (master) $

```

Worker Name	Status	Active	Processed	Failed	Succeeded	Retried	Load Average
celery@6dafb601ed9e	Online	0	0	0	0	0	0.62, 0.31, 0.25
celery@29ea6008c6da	Online	0	0	0	0	0	0.62, 0.31, 0.25
celery@85cb9336a3fb6	Online	0	0	0	0	0	0.62, 0.31, 0.25

In the DAG configuration file, it's now possible to separate each task for each specific node:

```

with DAG(dag_id='queue_dag', schedule_interval='0 0 * * *', default_args=default_args, catchup=False) as dag:

    t_1_ssd = BashOperator(task_id='t_1_ssd', bash_command='echo "I/O intensive task"', queue='worker_ssd')
    t_2_ssd = BashOperator(task_id='t_2_ssd', bash_command='echo "I/O intensive task"', queue='worker_ssd')
    t_3_ssd = BashOperator(task_id='t_3_ssd', bash_command='echo "I/O intensive task"', queue='worker_ssd')
    t_4_cpu = BashOperator(task_id='t_4_cpu', bash_command='echo "CPU intensive task"', queue='worker_cpu')
    t_5_cpu = BashOperator(task_id='t_5_cpu', bash_command='echo "CPU intensive task"', queue='worker_cpu')
    t_6_spark = BashOperator(task_id='t_6_spark', bash_command='echo "Spark dependency task"', queue='worker_spark')

    task_7 = DummyOperator(task_id='task_7')

    [t_1_ssd, t_2_ssd, t_3_ssd, t_4_cpu, t_5_cpu, t_6_spark] >> task_7

```

Pools

In Airflow, pools and priority_weights are mechanisms to manage task parallelism and prioritize execution.

Pools limit the number of tasks that can run concurrently within a specific resource group, preventing overload on external systems or shared resources. Each task is assigned to a pool, and the number of available "slots" determines how many tasks from that pool can run at the same time. priority_weights define the execution priority of tasks within the same pool or DAG. Tasks with higher weights are picked first by the scheduler when multiple tasks are eligible to run. Together, these tools help balance workload distribution and ensure critical tasks are handled in an efficient way.

The screenshot shows the Airflow web interface with a teal header containing navigation links: Airflow, DAGs, Data Profiling, Browse, Admin, Docs, and About. Below the header, there are two tabs: 'List' and 'Create'. The 'Create' tab is active, displaying a form to create a new pool. The form has three main fields: 'Pool *' with the value 'forex_api_pool', 'Slots' with the value '1', and 'Description' with the text 'pool to limit the number of requests to the API'. At the bottom of the form, there are four buttons: 'Save' (dark teal), 'Save and Add Another' (light teal), 'Save and Continue Editing' (light teal), and 'Cancel' (red).


```
# get forex rates of JPY and push them into XCOM
get_forex_rate_USD = SimpleHttpOperator(
    task_id='get_forex_rate_USD',
    method='GET',
    pool = 'forex_api_pool',
    http_conn_id='forex_api',
    endpoint='/latest?base=USD',
    xcom_push=True
)
```

Note that it was also applied to every other task that was using the API as well.

Kubernetes Reminder

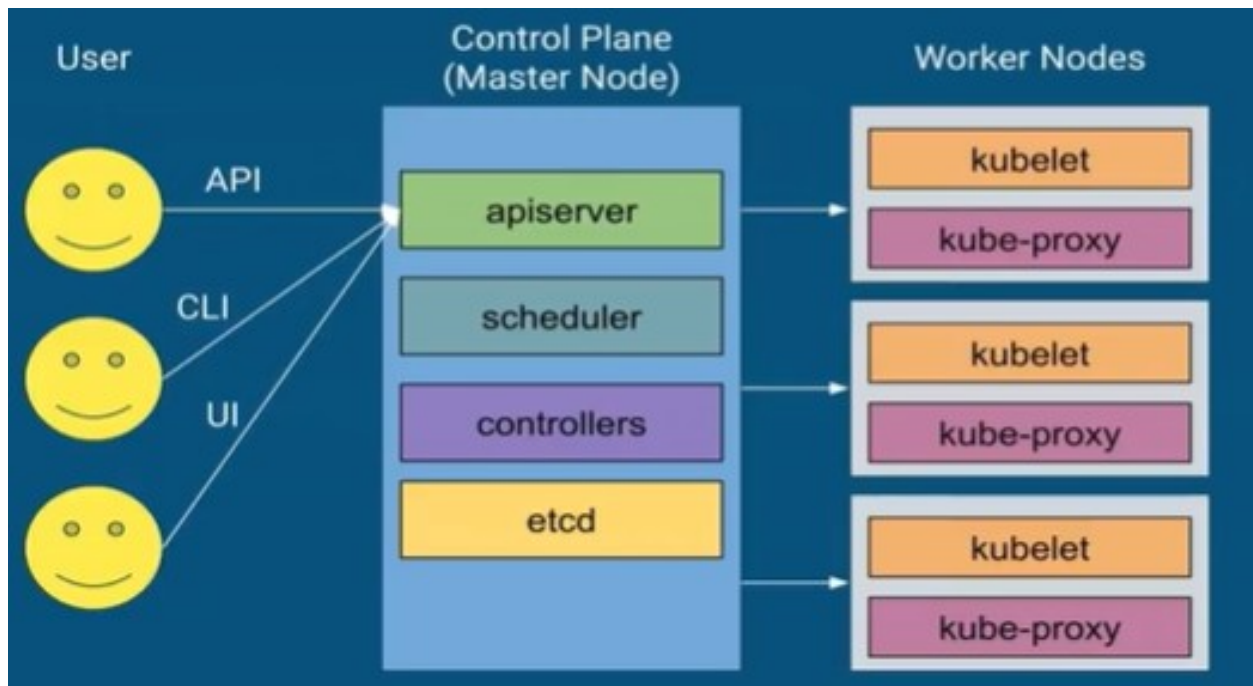
Kubernetes is extremely powerful:

- Open source project founded by Google in 2014.
- Container orchestrator.
- Kubernetes automates sysadmin tasks:
- Upgrading servers, installing security patches, configuring networks, running backups, failover, monitoring.
- Zero-downtime deployments with rolling updates.
- Provides facilities to implement CD practices: Canary / Blue-green.
- Supports autoscaling (up and down).
- Redundancy and failover built in, applications are more reliable and resilient.

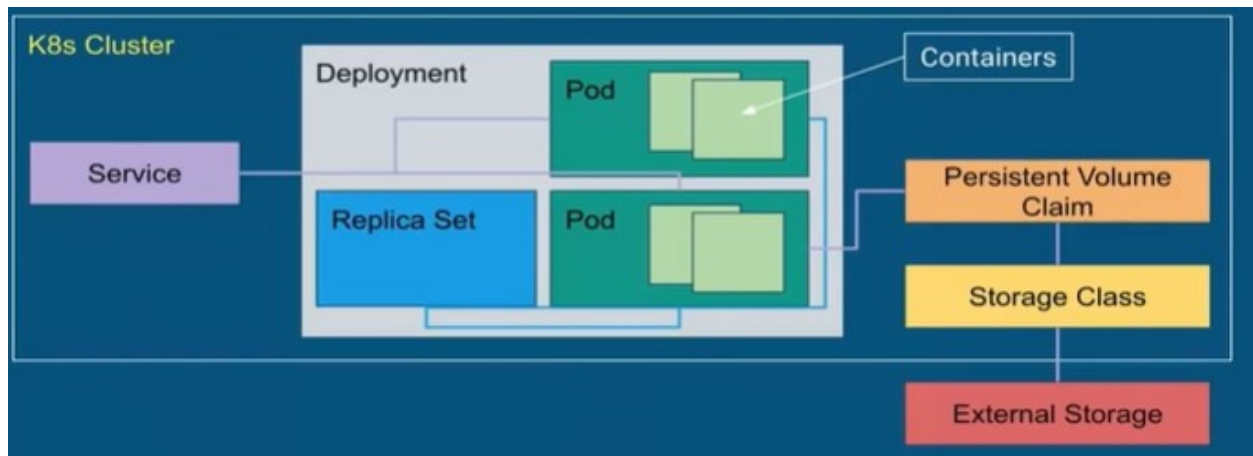
But there are some drawbacks:

- It's hard to correctly set up a K8s (kubernetes cluster).
- Hard to maintain.
- Has a steep learning curve.

Kubernetes Cluster Architecture



Important Kubernetes Objects



Scaling Airflow with Kubernetes Executors

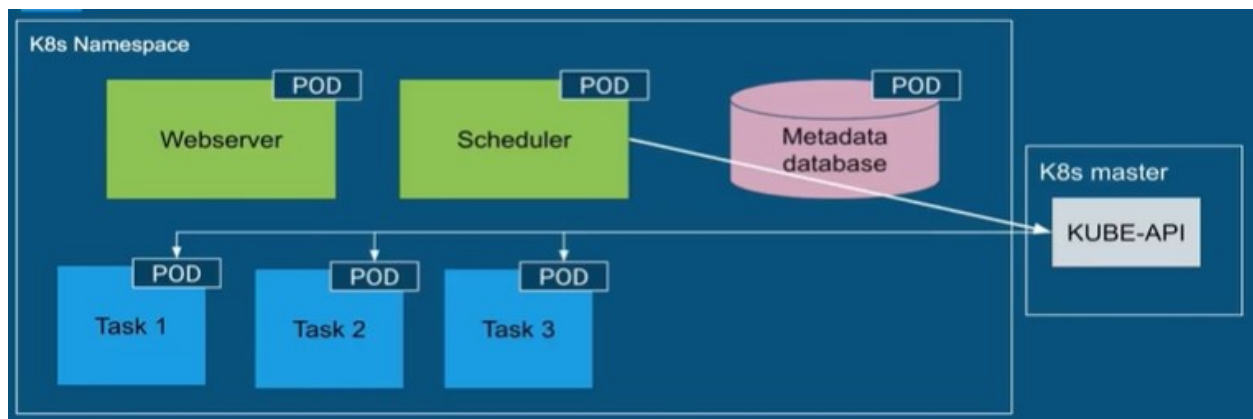
Drawbacks of the Celery Executor

- Need to set up tier applications (RabbitMQ, Celery, Flower).
- Deal with missing dependencies.
- Wasting resources: Airflow workers stay idle when no workload.
- Worker nodes are not as resilient as you think.

Nonetheless, it stays a widely used executor in production.

What is the Kubernetes Executor?

- Runs your tasks on Kubernetes.
- One task = One Pod.
- Task-level Pod configuration.
- Expands and shrinks your cluster according to the workload.
- Pods run to completion.
- Scheduler subscribes to Kubernetes event stream.
- DAG distribution:
 - Git clone with init-container for each Pod.
 - Mount volume with DAGs.
 - Build image with the DAG codes.



Setting up a 3-nodes Kubernetes Cluster with Vagrant and Rancher

Getting started with Vagrant

Firstly, it's needed to install the "vagrant-vbguest" and "vagrant-scp" plugins:

```
Marc@MacBook-Pro ~/airflow-materials/airflow-section-5/vagrant-files (master) $  
vagrant plugin install vagrant-vbguest vagrant-scp
```

Before any other commands, it's important to check if the console is being run at the directory that contains the Vagrantfile, which contains configuration arguments for it's instalation. After this is checked, the following command runs vagrant:

```
vagrant up
```

After it, the Virtual Machines (VMs) are already going to be running. To check it's state, run `vagrant status`:

```
Marc@MacBook-Pro ~/airflow-materials/airflow-section-5/vagrant-files (master) $  
vagrant status  
Current machine states:  
  
master                running (virtualbox)  
worker-1              running (virtualbox)  
worker-2              running (virtualbox)  
  
This environment represents multiple VMs. The VMs are all listed  
above with their current state. For more information about a specific  
VM, run `vagrant status NAME`.
```

SSH is used to connect into a node:

```
vagrant ssh <node_name>
```

After being logged in, the command `kubectl get nodes` is used to have an overview running nodes:

```
Marc@MacBook-Pro ~/airflow-materials/airflow-section-5/vagrant-files (master) $  
vagrant ssh master  
Last login: Tue Jan 21 18:41:10 2020  
[vagrant@master ~]$ kubectl get nodes  
NAME                STATUS    ROLES    AGE   VERSION  
master.cluster.com  Ready    master   13m   v1.15.4  
worker-1.cluster.com Ready    <none>   8m3s  v1.15.4  
worker-2.cluster.com Ready    <none>   2m11s  v1.15.4
```

To log out, simply type `logout` when connected to the VM through SSH.

Getting started with Rancher

Rancher will be used to manage the Kubernetes cluster through a friendly interface.

To start the rancher container:

```
docker run -d --restart=unless-stopped -p 80:80 -p 443:443 --name  
rancher rancher/rancher:v2.3.2
```

This exposes the ports 80 and 443 for it.

```

PS C:\Users\pedro> docker run -d --restart=unless-stopped -p 80:80 -p 443:443 --name rancher rancher/r
ancher:v2.3.2
Unable to find image 'rancher/rancher:v2.3.2' locally
v2.3.2: Pulling from rancher/rancher
c58094023a2e: Download complete
8a37a3d9d32f: Download complete
9acf582a7992: Download complete
11048ebae908: Download complete
322f0c253a60: Download complete
ff331cbe510b: Download complete
e1d7887879ba: Download complete
bed4e005ec0d: Download complete
22e816666fd6: Download complete
5a5441e6019b: Download complete
e403b6985877: Download complete
079b6d2a1e53: Download complete
883600f5c6cf: Download complete
74a2e9817745: Download complete
Digest: sha256:c1f125266d89ec64a60385c0d2b63869ce528881315a76387f53b87dc08815a8
Status: Downloaded newer image for rancher/rancher:v2.3.2
869031ea8452c725fa0d9cee656c7943ede5c5e17109bf89a7b4daa54d1c6c7d
PS C:\Users\pedro>

```

```

PS C:\Users\pedro> docker ps

```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
869031ea8452	rancher/rancher:v2.3.2	rancher	"entrypoint.sh"	30 seconds ago	Up 29 seconds	0.0.0.0:80->80/tcp, 0.0.0.0:443->443/tcp

```

PS C:\Users\pedro> |

```

Now it can be accessed through the web browser, at localhost in the specified ports:

Not secure https://localhost/update-password

Welcome to Rancher

The first order of business is to set a strong password for the default `admin` user.

☒ Allow collection of anonymous statistics [Learn More](#)

Current Password *

☒ Set a specific password to use:

New Password

Confirm Password

☐ Use a new randomly generated password:

Continue



Global ▾ Clusters Apps Users Settings Security ▾ Tools ▾



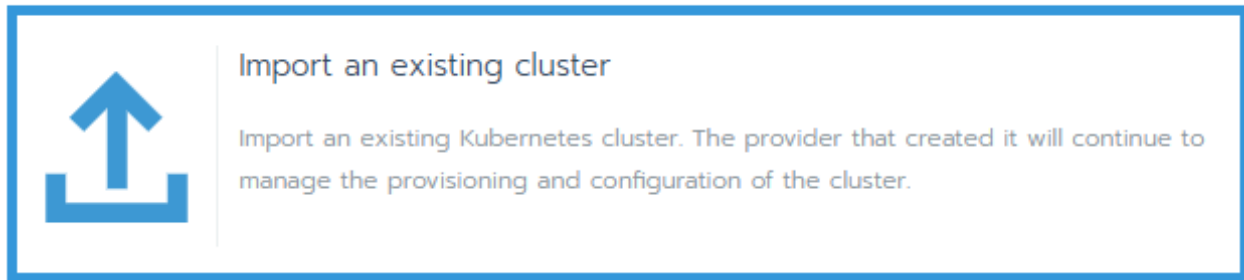
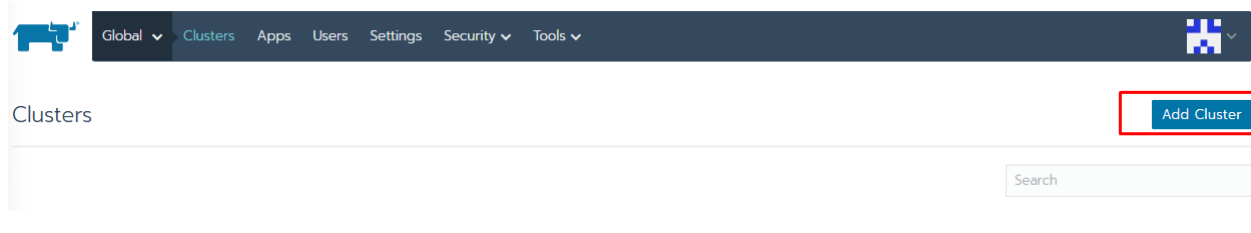
Clusters Add Cluster

State ▾ Cluster Name ▾ Provider ▾ Nodes ▾ CPU ▾ RAM ▾

Search

There are no clusters defined.

Adding a new cluster



Add Cluster - Import

Cluster Name * Add a Description

airflow

Member Roles
Control who has access to the cluster and what permission they have to change it.

Labels & Annotations
Configure labels and annotations for the cluster. None

Create Cancel

Now, the following command needs to be executed in the master node in order to import the cluster into Rancher:

Add Cluster - Import

Note: If you want to import a Google Kubernetes Engine (GKE) cluster (or any cluster that does not supply you with a kubectl configuration file with the ClusterRole **cluster-admin** bound to it), you need to bind the ClusterRole **cluster-admin** using the command below.

Replace **[USER_ACCOUNT]** with your Google account address (you can retrieve this using **gcloud config get-value account**). If you are not importing a Google Kubernetes Engine cluster, replace **[USER_ACCOUNT]** with the executing user configured in your kubectl configuration file.

```
kubectl create clusterrolebinding cluster-admin-binding --clusterrole cluster-admin --user [USER_ACCOUNT]
```

Run the kubectl command below on an existing Kubernetes cluster running a supported Kubernetes version to import it into Rancher:

```
kubectl apply -f https://192.168.56.1/v3/import/97vkgz22wkdxbjkd4jfr9g8vk9pqbxnvr7tdh7pkzn87cpxlg2.yaml
```

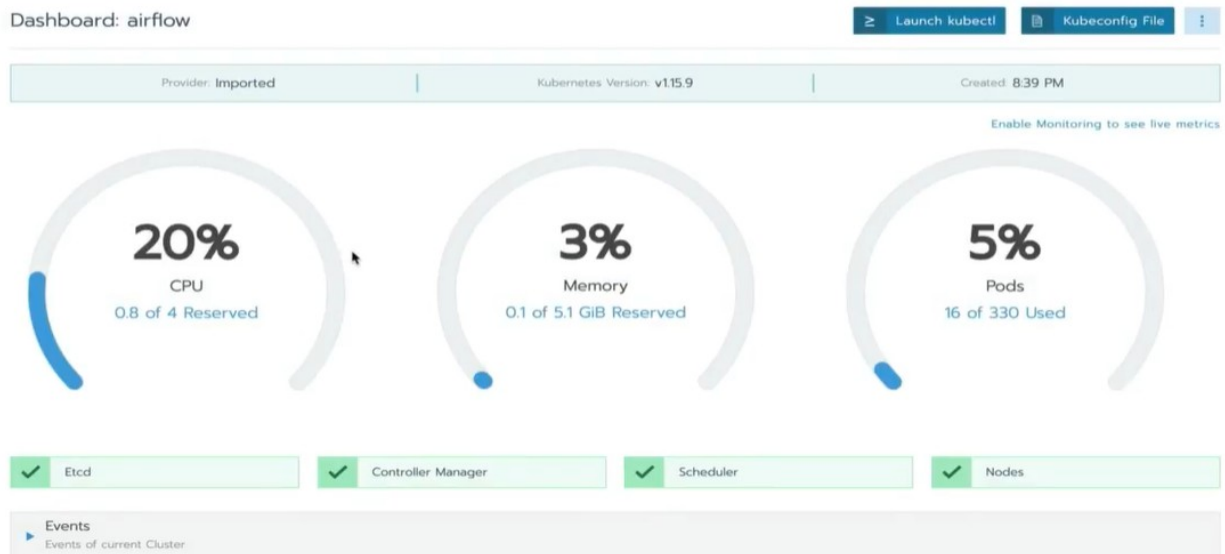
If you get an error about 'certificate signed by unknown authority' because your Rancher installation is running with an untrusted/self-signed SSL certificate, run the command below instead to bypass the certificate check:

```
curl --insecure -sL https://192.168.56.1/v3/import/97vkgz22wkdxbjkd4jfr9g8vk9pqbxnvr7tdh7pkzn87cpxlg2.yaml | kubectl apply -f -
```

After it, it should have been successfully imported into Rancher:



Now, system resources can be monitored through Rancher's dashboard:



Improving your DAG with advanced concepts

In the case of many parallel tasks, they can be grouped up in a single task, using the `SubDagOperator`, which calls the `Factory Function`. Note that this does not alter any dag configuration, it's used only for better visual clarity for diagrams.

Practicing

Firstly, executing airflow:

```
Marc@macbook-pro ~/airflow-materials/airflow-section-6 (master) $ pwd
/Users/Marc/airflow-materials/airflow-section-6
Marc@macbook-pro ~/airflow-materials/airflow-section-6 (master) $ docker-compose -f docker-compose-CeleryExecutor.yml up -d
Creating network "airflow-section-6_default" with the default driver
Creating airflow-section-6_postgres_1 ... done
Creating airflow-section-6_redis_1 ... done
Creating airflow-section-6_flower_1 ... done
Creating airflow-section-6_webserver_1 ... done
Creating airflow-section-6_scheduler_1 ... done
Creating airflow-section-6_worker_1 ...
```

```

DAG_NAME="test_subdag"

default_args = {
    'owner': 'Airflow',
    'start_date': airflow.utils.dates.days_ago(2)
}

with DAG(dag_id=DAG_NAME, default_args=default_args, schedule_interval="@once") as dag:
    start = DummyOperator(
        task_id='start'
    )

    subdag_1 = SubDagOperator(
        task_id='subdag-1',
        subdag=factory_subdag(DAG_NAME, 'subdag-1', default_args),
        executor=SequentialExecutor()
    )

    some_other_task = DummyOperator(
        task_id='check'
    )

    subdag_2 = SubDagOperator(
        task_id='subdag-2',
        subdag=factory_subdag(DAG_NAME, 'subdag-2', default_args),
        executor=SequentialExecutor()
    )

    end = DummyOperator(
        task_id='final'
    )

    start >> subdag_1 >> some_other_task >> subdag_2 >> end

```

*Note that the subdags can run in parallel if the operator is changed from SequentialOperator to CeleryExecutor.

Now, executing the task and waiting for it to be completed:

	DAG	Schedule	Owner	Recent Tasks	Last Run	DAG Runs	Links
☒	☐ branch_dag	Once	Airflow				
☒	☐ deadlock_subdag	Once	Airflow				
☒	☐ externaltasksensor_dag	Daily	airflow				
☒	☐ sleep_dag	Daily	airflow				
☒	☐ template_dag	Daily	Airflow				
☒	☒ test_subdag	Once	Airflow		2020-01-20 00:00		
☒	☐ triggerdagop_controller_dag	Once	airflow				
☒	☐ triggerdagop_target_dag	None	Airflow				
☒	☐ trigger_rule_dag	Daily	Airflow				
☒	☐ xcom_dag	Once	Airflow				
☒	☐ xcom_dag_big	Once	Airflow				

Showing 1 to 11 of 11 entries

Now, the subtask has been already created:



And by clicking on it, its possible to visualize all the tasks included in it:

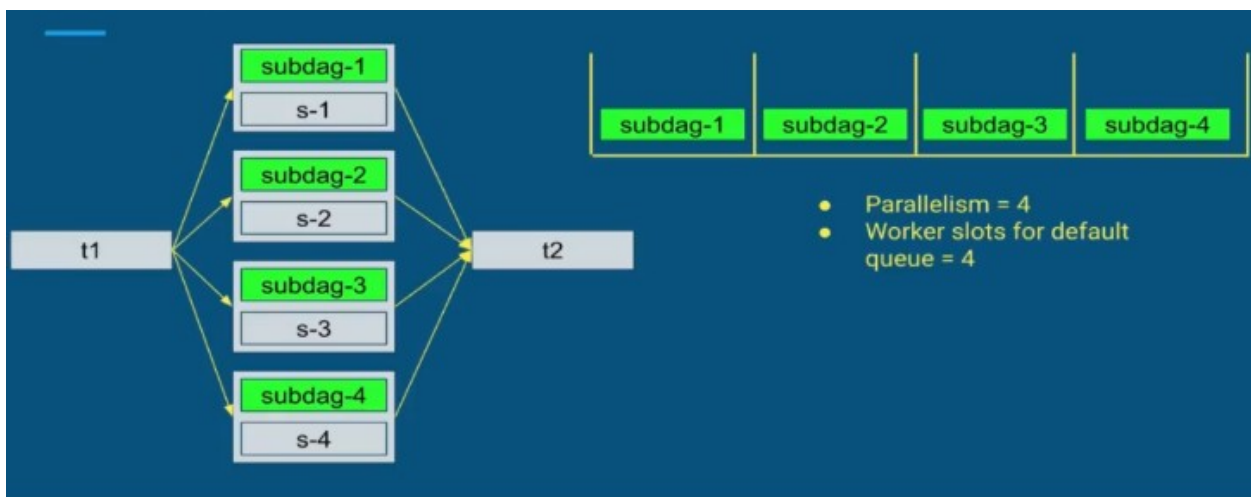
SUBDAG: test_subdag.subdag-1

[← Back to test_subdag](#)
[Graph View](#)
[Tree View](#)

success
Base date: 2020-01-20 00:00:01
Number of n

DummyOperator

SubDag DeadLock



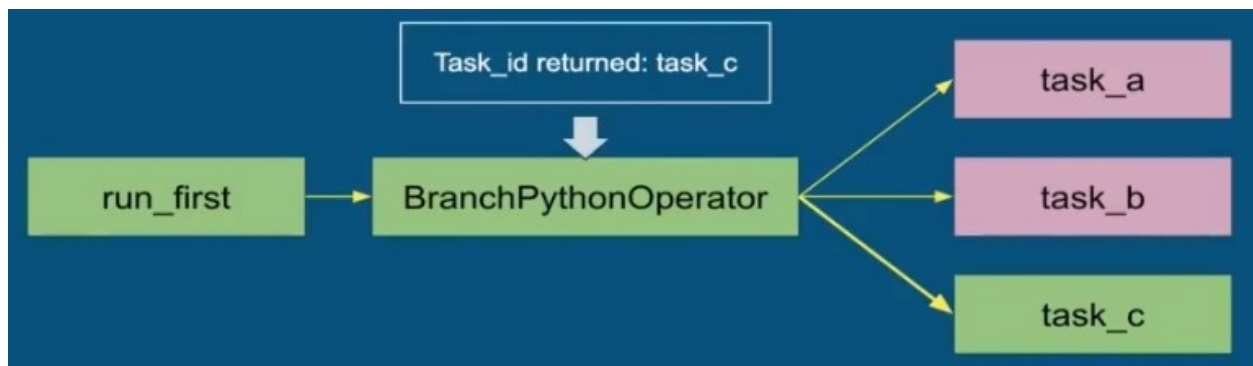
DeadLock happens because each SubDag is treated as a task, so when parallelism is set to 4, there are 4 subdags to be run and each subdag contains a task, the subdags will take all the slots for parallelism, making the execution of its tasks not possible, creating a deadlock.

A queue can be used to fix it, but to make things easy, just simply use the sequential executor or simply not use subdags at all.

Making different paths in your DAGs with Branching

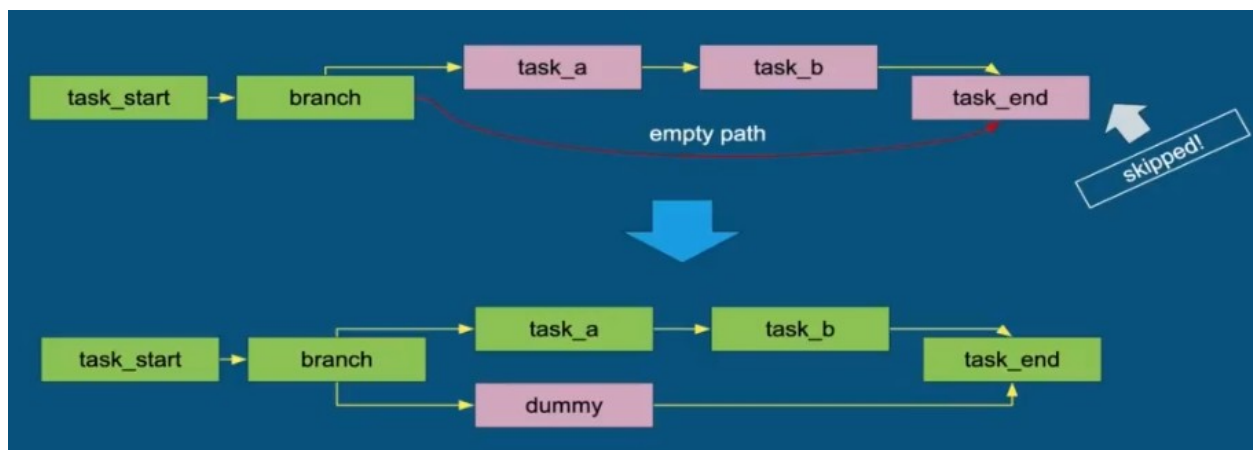
Branching is the mechanism that allows your DAG to choose between different paths according to the result of a specific task. Can be referenced as BranchPythonOperator.

It works by returning a task_id, which will be used to execute an specific task after its completed:



Important Notes

It's not recommended to be skipping tasks, so if at some point it turns out to be useful, the dag needs to have another path to get into the end of the dag run. It can be done by creating a dummy task directly connected to the end of the task by using the branch operator:



Practicing

branch_dag.py:

```

IP_GEOLOCATION_APIS = {
    'ip-api': 'http://ip-api.com/json/',
    'ipstack': 'https://api.ipstack.com/',
    'ipinfo': 'https://ipinfo.io/json'
}

# Try to get the country_code field from each API
# If given, the API is returned and the next task corresponding
# to this API will be executed
def check_api():
    for api, link in IP_GEOLOCATION_APIS.items():
        r = requests.get(link)
        try:
            data = r.json()
            if data and 'country' in data and len(data['country']):
                return api
        except ValueError:
            pass
    return 'none'

with DAG(dag_id='branch_dag',
        default_args=default_args,
        schedule_interval="@once") as dag:

    # BranchPythonOperator
    # The next task depends on the return from the
    # python function check_api
    check_api = BranchPythonOperator(
        task_id='check_api',
        python_callable=check_api
    )

    none = DummyOperator(
        task_id='none'
    )

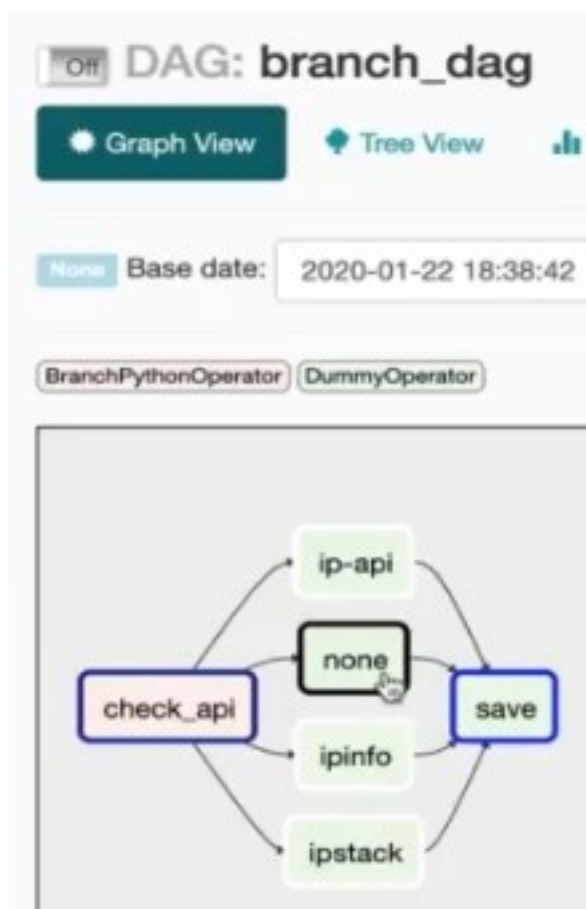
    save = DummyOperator(task_id='save')

    check_api >> none >> save

```

Clicking on this task in airflow:

DAGs							
Search:							
	DAG	Schedule	Owner	Recent Tasks	Last Run	DAG Runs	Links
	branch_dag	@once	Airflow				
	deadlock_subdag	@once	Airflow				
	externaltasksensor_dag	@daily	airflow				
	sleep_dag	@daily	airflow				
	template_dag	@daily	Airflow				
	test_subdag	@once	Airflow				
	triggerdagop_controller_dag	@once	airflow				
	triggerdagop_target_dag	None	Airflow				
	trigger_rule_dag	@daily	Airflow				
	xcom_dag	@once	Airflow				
	xcom_dag_big	@once	Airflow				



After the task is completed, it's possible to visualize that the only task that has been executed was the ip-api, which was returned by the branch operator:

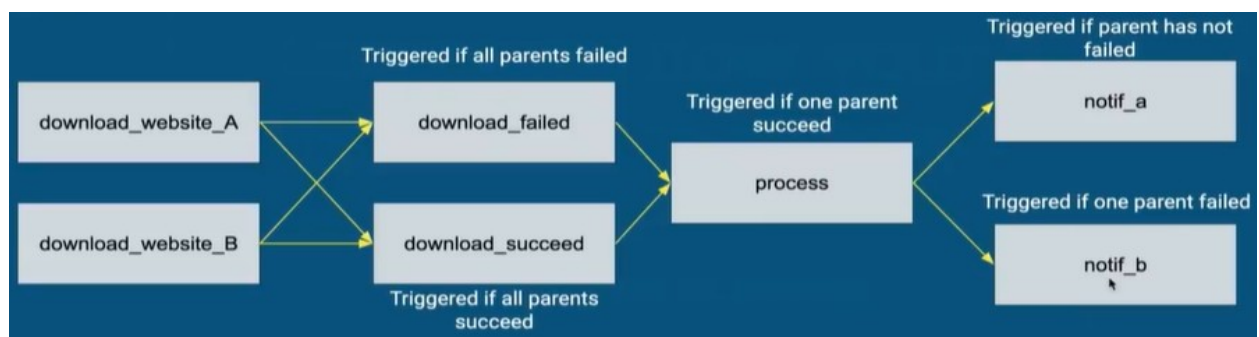
```
py:105} INFO - Exporting the following env vars:
0:00+00:00
20T00:00:00+00:00
py:114} INFO - Done. Returned value was: ip-api
INFO - Following branch ip-api
```

Trigger rules for your tasks

Trigger rules makes possible to decide which task will be executed based on the result of the previous task: if it was successful or not (all_success / all_failed); if at least one of the previous tasks were not failed (none_failed) and many other:

Practicing

For this practice, the following use case will be implemented:



Here, the code for the tasks, which shows what are the trigger rules for each of the tasks present in the dag.

```

with DAG(dag_id='trigger_rule_dag',
         default_args=default_args,
         schedule_interval="@daily") as dag:

    # Change the trigger rules according to the
    # dag from the video
    # all_success
    # all_failed
    # all_done
    # one_failed
    # one_success
    # none_failed
    # none_skipped

    download_website_a_task = PythonOperator(
        task_id='download_website_a',
        python_callable=download_website_a,
        trigger_rule="all_success"
    )

    download_website_b_task = PythonOperator(
        task_id='download_website_b',
        python_callable=download_website_b,
        trigger_rule="all_success"
    )

    download_failed_task = PythonOperator(
        task_id='download_failed',
        python_callable=download_failed,
        trigger_rule="all_success"
    )

    download_succeed_task = PythonOperator(
        task_id='download_succeed',
        python_callable=download_succeed,
        trigger_rule="all_success"
    )

    process_task = PythonOperator(
        task_id='process',
        python_callable=process,
        trigger_rule="all_success"
    )

    notif_a_task = PythonOperator(
        task_id='notif_a',
        python_callable=notif_a,
        trigger_rule="all_success"
    )

```

On airflow:



As it can be seen, there is no dependencies yet between each tasks. For this, some modifications needs to be taken into the python dag file:

```
101 |  
102 | [ download_failed, download_succeed ] >> process_task
```

Now, the applied dependencies can be seen in the graph view:



Setting up the dependencies to fit the use case above:

```

96     # Implement dependencies below
97     # ie:
98     # a >> b          : b depends on a
99     # [a b] >> c      : c depends on a and b
100    # ...
101
102    [ download_failed, download_succeed ] >> process_task
103    [ download_website_a_task, download_website_b_task ] >> download_failed_task
104    [ download_website_a_task, download_website_b_task ] >> download_succeed_task
105    process_task >> [ notif_a_task, notif_b_task ]

```

Now, the graph view should be the same as the use case:



Now that the dependencies are all set, its time to set the correct trigger rules for each task.

Setting up the trigger rules correctly

1. `download_failed_task` needs to have its trigger rule set to "all_failed".

```
download_failed_task = PythonOperator(  
    task_id='download_failed',  
    python_callable=download_failed,  
    trigger_rule="all_failed"  
)
```

1. `process_task` needs to have its trigger rule set to "one_success", since the task will either fail or succeed.

```
process_task = PythonOperator(  
    task_id='process',  
    python_callable=process,  
    trigger_rule="one_success"  
)
```

1. `notif_a_task` needs to have its trigger rule set to "none_failed".

```

notif_a_task = PythonOperator(
    task_id='notif_a',
    python_callable=notif_a,
    trigger_rule="none_failed"
)

```

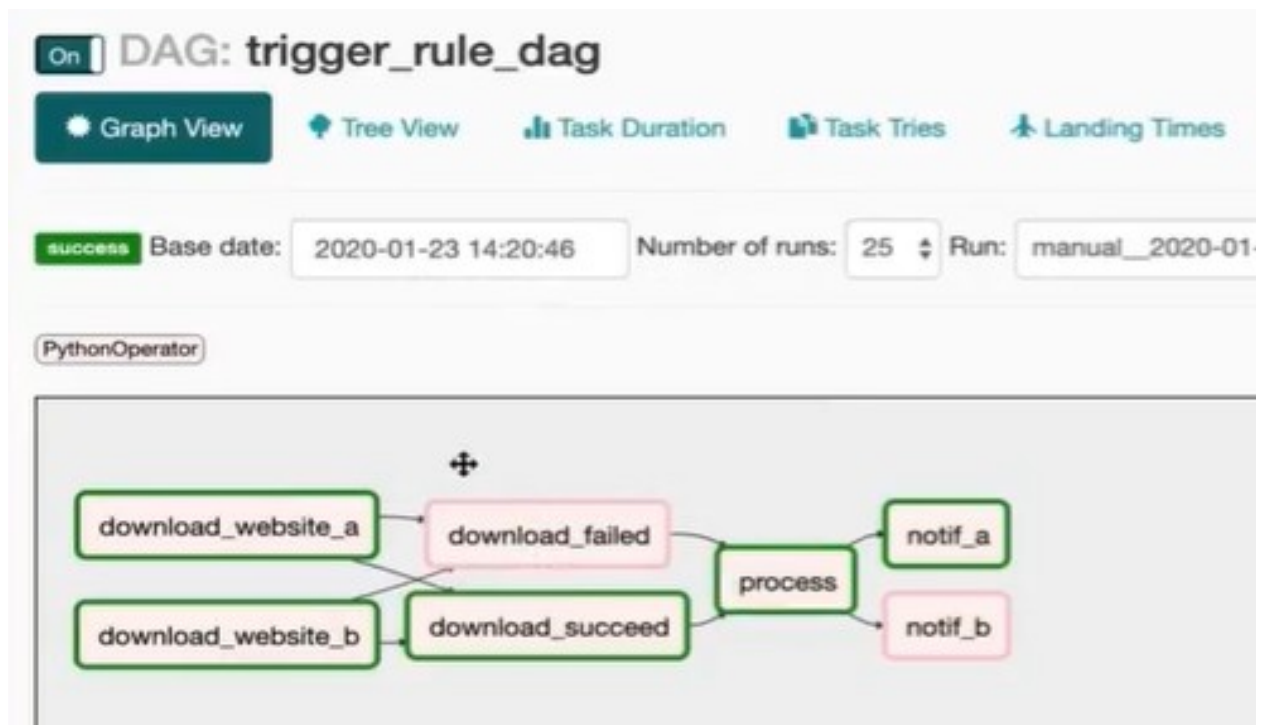
1. `notif_b_task` must be run as soon as at least one parent has failed, so the trigger rule should be set to "one_failed".

```

notif_b_task = PythonOperator(
    task_id='notif_b',
    python_callable=notif_b,
    trigger_rule="one_failed"
)

```

With that changed being applied, lets now trigger the dag:



Since both download tasks were successful, `download_failed` was skipped, the same applies to `notif_b`, since nothing has failed.

Templating your tasks

Templating the tasks are important to maintain consistent information throughout the output obtained through the execution of a dag in different instances in time.

```
with DAG(dag_id="template_dag", schedule_interval="@daily", default_args=default_args) as dag:

    t0 = BashOperator(
        task_id="t0",
        bash_command="echo {{ macros.ds_format(ts_nodash, '%Y%m%dT%H%M%S', '%Y-%m-%d-%H-%M') }}"

    t1 = BashOperator(
        task_id="generate_new_logs",
        bash_command="./scripts/generate_new_logs.sh",
        params={'filename': 'log.csv'})

    t2 = BashOperator(
        task_id="logs_exist",
        bash_command="test -f " + TEMPLATED_LOG_DIR + "log.csv",
    )

    t3 = PythonOperator(
        task_id="process_logs",
        python_callable=process_logs_func,
        provide_context=True,
        templates_dict={'log_dir': TEMPLATED_LOG_DIR},
        params={'filename': 'log.csv'}
    )

    t0 >> t1 >> t2 >> t3
```

The task t0 shows the current date of execution, formatting it using a macro.

The task t1 shows executes the script "generate_new_logs.sh" and saves its output in a csv file.

The task t2 checks if a file named log.csv exists in the directory represented by the variable.

The task t3 saves the csv file into the system.

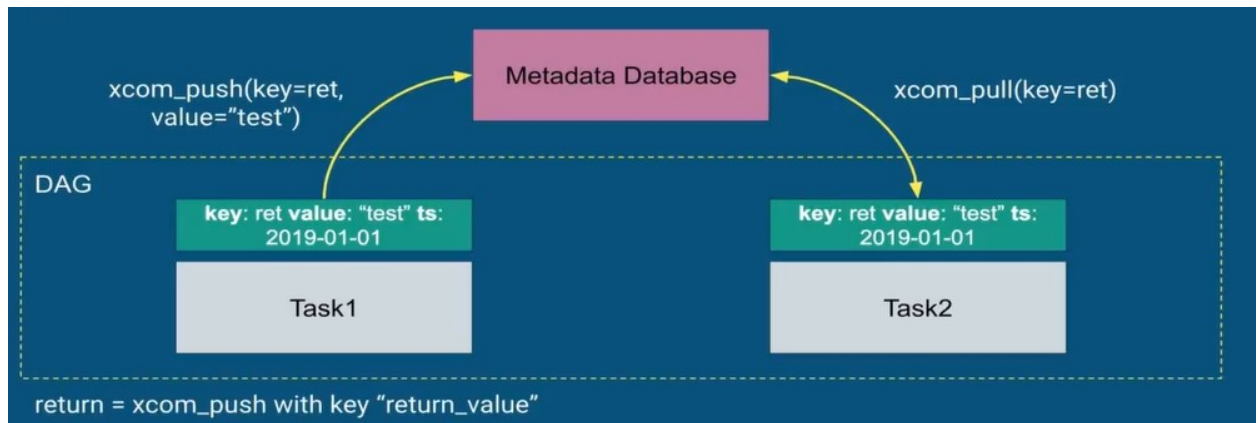
With this, data can now have more consistency, setting new variables accordingly to dynamic dates, useful for regular SQL searches, for example.

Sharing data between tasks with XCOM

XCOM allows multiple tasks to exchange messages between them, using keys, values and timestamps.

To push some data into the metadata database, the function `xcom_push(key=ret, value="test")` can be used (timestamp is the date the data was pushed into the database).

After a task has pushed data into the database, another task can access the data using the function `xcom_pull(key=ret)`, specifying the key of the desired data:



Practicing

```
xcom_dag.py 3 X
airflow-section-6 > mnt > airflow > dags > xcom_dag.py > get_next_task

6
7     args = {
8         'owner': 'Airflow',
9         'start_date': airflow.utils.dates.days_ago(1),
10    }
11
12    def push_xcom_with_return():
13        return 'my_returned_xcom'
14
15    def get_pushed_xcom_with_return(**context):
16        print(context['ti'].xcom_pull(task_ids='t0'))
17
18    def push_next_task(**context):
19        context['ti'].xcom_push(key='next_task', value='t3')
20
21    def get_next_task(**context):
22        return context['ti'].xcom_pull(key='next_task')
23
24    def get_multiple_xcoms(**context):
25        print(context['ti'].xcom_pull(key=None, task_ids=['t0', 't2']))
26
27    with DAG(dag_id='xcom_dag', default_args=args, schedule_interval="@once") as dag:
28
29        t0 = PythonOperator(
30            task_id='t0',
31            python_callable=push_xcom_with_return
32        )
33
34        t1 = PythonOperator(
35            task_id='t1',
36            provide_context=True,
37            python_callable=get_pushed_xcom_with_return
38        )
39
40        t2 = PythonOperator(
41            task_id='t2',
42            provide_context=True,
43            python_callable=push_next_task
44        )
45
46        branching = BranchPythonOperator(
47            task_id='branching',
48            provide_context=True,
49            python_callable=get_next_task,
50        )
51
52        t3 = DummyOperator(task_id='t3')
```

```

t4 = DummyOperator(task_id='t4')

t5 = PythonOperator(
    task_id='t5',
    trigger_rule='one_success',
    provide_context=True,
    python_callable=get_multiple_xcoms
)

t6 = BashOperator(
    task_id='t6',
    bash_command="echo value from xcom: {{ ti.xcom_pull(key='next_task') }}"
)

t0 >> t1
t1 >> t2 >> branching
branching >> t3 >> t5 >> t6
branching >> t4 >> t5 >> t6

```

In this dag, there are some tasks that push/pull data to/from the database, as well a branch operator, to decide which task will be executed based on the data pulled using xcom.

By triggering this dag, some xcoms are pushed:

Xcoms

List (2)	Create	Add Filter	With selected	Search: key, timestamp			
		Key	Value	Timestamp	Execution Date	Task Id	Dag Id
☐	 	return_value	my_returned_xcom	2020-01-23 17:46:51.203315+00:00	2020-01-22 00:00:00+00:00	t0	xcom_dag
☐	 	next_task	t3	2020-01-23 17:47:10.913450+00:00	2020-01-22 00:00:00+00:00	t2	xcom_dag

TriggerDagRunOperator or when your DAG controls another DAG

Its used to trigger a dag from another dag. Its useful to modularize tasks execution so a DAG doesnt contain an insane amount of tasks, which would make its execution confusing and hard to maintain. Checking the following use case:



TriggerDagRunOperator parameters:

- trigger_dag_id
- python_callable
- params

Important notes:

- Controller does not wait for target to finish
- Controller and target are independent
- No visualization from the controller neither history
- Both DAGs must be scheduled
- Target DAG schedule interval = None

Practicing

Below, there are the files for the dag configuration for the controller and target dag.


```

triggerdagop_controller_dag.py x triggerdagop_target_dag.py 2
airflow-section-6 > mnt > airflow > dags > triggerdagop_controller_dag.py > ...
1 import pprint as pp
2 import airflow.utils.dates
3 from airflow import DAG
4 from airflow.operators.dagrun_operator import TriggerDagRunOperator
5 from airflow.operators.dummy_operator import DummyOperator
6
7 default_args = {
8     "owner": "airflow",
9     "start_date": airflow.utils.dates.days_ago(1)
10 }
11
12 def conditionally_trigger(context, dag_run_obj):
13     if context['params']['condition_param']:
14         dag_run_obj.payload = {
15             'message': context['params']['message']
16         }
17     pp.pprint(dag_run_obj.payload)
18     return dag_run_obj
19
20 with DAG(dag_id="triggerdagop_controller_dag", default_args=default_args, schedule_interval="@once") as dag:
21     trigger = TriggerDagRunOperator(
22         task_id="trigger_dag",
23         trigger_dag_id="triggerdagop_target_dag",
24         provide_context=True,
25         python_callable=conditionally_trigger,
26         params={
27             'condition_param': True,
28             'message': 'Hi from the controller'
29         },
30     )
31
32     last_task = DummyOperator(task_id="last_task")
33
34     trigger >> last_task

```

```

triggerdagop_controller_dag.py 2 triggerdagop_target_dag.py 2 X
airflow-section-6 > mnt > airflow > dags > triggerdagop_target_dag.py > ...
1 import airflow.utils.dates
2 from airflow.models import DAG
3 from airflow.operators.bash_operator import BashOperator
4 from airflow.operators.python_operator import PythonOperator
5
6 default_args = {
7     "start_date": airflow.utils.dates.days_ago(1),
8     "owner": "Airflow"
9 }
10
11 def remote_value(**context):
12     print("Value {} for key=message received from the controller DAG".format(context["dag_run"].conf["message"]))
13
14 with DAG(dag_id="triggerdagop_target_dag", default_args=default_args, schedule_interval=None) as dag:
15
16     t1 = PythonOperator(
17         task_id="t1",
18         provide_context=True,
19         python_callable=remote_value,
20     )
21
22     t2 = BashOperator(
23         task_id="t2",
24         bash_command='echo Message: {{ dag_run.conf["message"] if dag_run else "" }}')
25
26     t3 = BashOperator(
27         task_id="t3",
28         bash_command="sleep 30"
29 )

```

Triggering the controller and target dag and watching what happens to the target dag:

	triggerdagop_controller_dag	@once	airflow			2020-01-22 00:00	
	triggerdagop_target_dag	None	Airflow			2020-01-23 20:15	

Looking at the logs in the target dag to comprehend what's happened:

For the task t1, it completes its execution by showing the information passed from the controller dag:

```

[2020-01-23 20:16:05,575] {python_operator.py:105} INFO - Exporting the following env vars:
AIRFLOW_CTX_DAG_ID=triggerdagop_target_dag
AIRFLOW_CTX_TASK_ID=t1
AIRFLOW_CTX_EXECUTION_DATE=2020-01-23T20:15:58.064183+00:00
AIRFLOW_CTX_DAG_RUN_ID=trig__2020-01-23T20:15:58.030014+00:00
[2020-01-23 20:16:05,575] {logging_mixin.py:95} INFO - Value Hi from the controller for key=message received from the controller DAG
[2020-01-23 20:16:05,575] {python_operator.py:114} INFO - Done. Returned value was: None
[2020-01-23 20:16:07,744] {logging_mixin.py:95} INFO - [ [34m2020-01-23 20:16:07,743 [0m] { [34mlocal_task_job.py: [0m105} INFO [0m - Task e

```

For the task t2, the same happens:

```

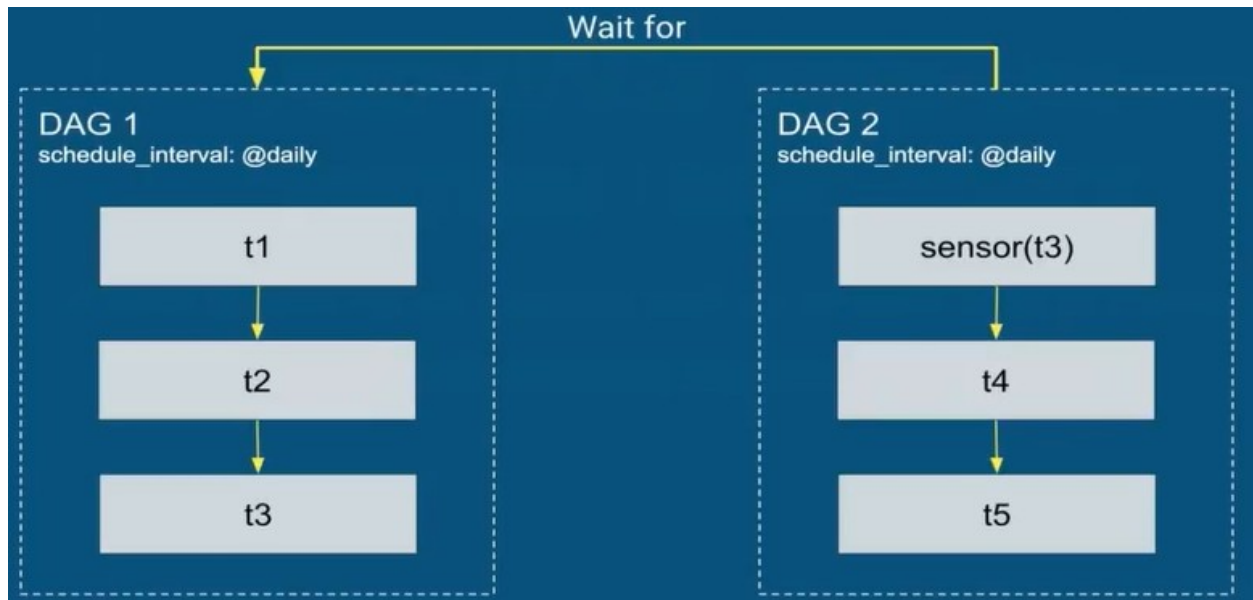
AIRFLOW_CTX_EXECUTION_DATE=2020-01-23T20:15:58.064183+00:00
AIRFLOW_CTX_DAG_RUN_ID=trig__2020-01-23T20:15:58.030014+00:00
[2020-01-23 20:16:05,553] {bash_operator.py:105} INFO - Temporary script location: /tmp/airflowtmp_a11ldyz/t2yy870r8o
[2020-01-23 20:16:05,553] {bash_operator.py:115} INFO - Running command: echo Message: Hi from the controller
[2020-01-23 20:16:05,566] {bash_operator.py:124} INFO - Output:
[2020-01-23 20:16:05,568] {bash_operator.py:128} INFO - Message: Hi from the controller
[2020-01-23 20:16:05,568] {bash_operator.py:132} INFO - Command exited with return code 0
[2020-01-23 20:16:07,836] {logging_mixin.py:95} INFO - [ [34m2020-01-23 20:16:07,836 [0m] { [34mlocal_task_job.py: [0m105} IN

```

Thus, making the communication between DAGs successful.

Dependencies between your dags with the ExternalTaskSensor

It allows to wait for an external task (in another dag) to be completed.



So the DAG 2 will only start running its first task when the DAG 1 has already completed all of its tasks.

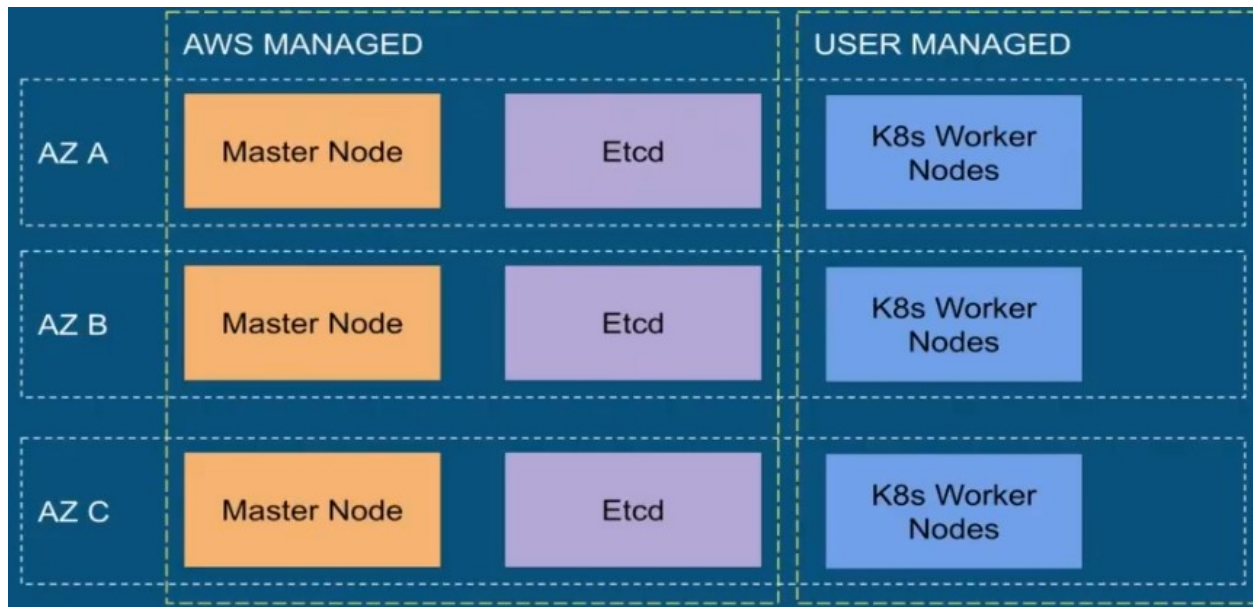
Important Notes

- Keep the same schedule between your DAGs
- Or use execution_delta or execution_date_fn parameters
- ExternalTaskSensor is different from TriggerDagRunOperator
 - Using both usually leads to getting the sensor stuck
- Mode "poke" used by default
 - "reschedule" if needed

Deploying Airflow on AWS EKS with Kubernetes Executors and Rancher

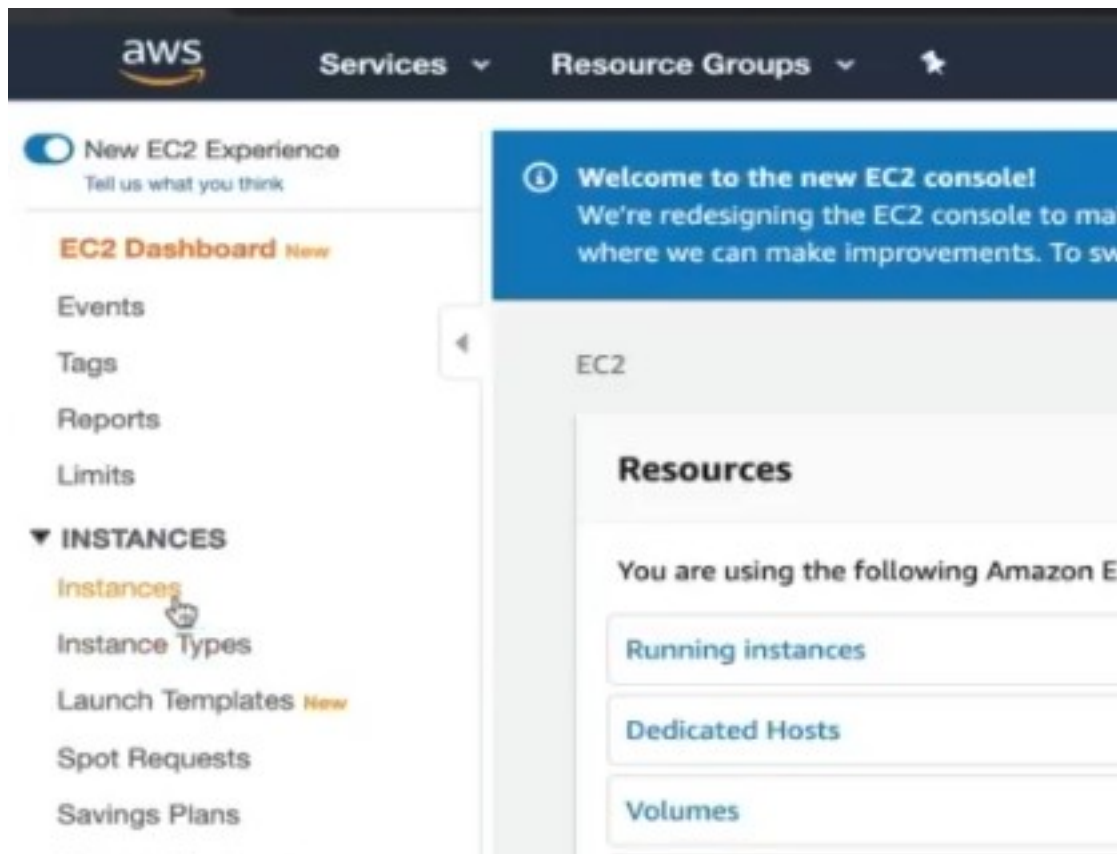
Overview:

- AWS EKS (Elastic Kubernetes Service)
- Makes it easy to deploy, manage, and scale containerized applications
- Orchestrate your containers
- Still need to configure subnets, VPS, IAM roles, SSH keys...
- Integrated with AWS services (ECR, ELB, IAM, ...)



Setting up an EC2 instance for Rancher

Once logged in the AWS console, click on "instances" and then



Then, an instance type needs to be chosen. Note that because of Rancher, it has to have at least 2GB Ram:

Step 2: Choose an Instance Type

Amazon EC2 provides a wide selection of instance types optimized to fit different use cases. Instances are virtual servers that can run applications. They have varying combinations of CPU, memory, storage, and networking capacity, and give you the flexibility to choose the appropriate mix of resources for your applications. [Learn more](#) about instance types and how they can meet your computing needs.

Filter by: **All instance types** **Current generation** **Show/Hide Columns**

Currently selected: t2.micro (Variable ECUs, 1 vCPUs, 2.5 GHz, Intel Xeon Family, 1 GiB memory, EBS only)

	Family	Type	vCPUs	Memory (GiB)	Instance Storage (GiB)	EBS-Optimized Available	Network Performance	IPv6 Support
☐	General purpose	t2.nano	1	0.5	EBS only	-	Low to Moderate	Yes
☒	General purpose	t2.micro	1	1	EBS only	-	Low to Moderate	Yes
☐	General purpose	t2.small	1	2	EBS only	-	Low to Moderate	Yes
☐	General purpose	t2.medium	2	4	EBS only	-	Low to Moderate	Yes
☐	General purpose	t2.large	2	8	EBS only	-	Low to Moderate	Yes
☐	General purpose	t2.xlarge	4	16	EBS only	-	Moderate	Yes
☐	General purpose	t2.2xlarge	8	32	EBS only	-	Moderate	Yes
☐	General purpose	t3a.nano	2	0.5	EBS only	Yes	Up to 5 Gigabit	Yes

[Cancel](#) [Previous](#) [Review and Launch](#) [Next: Configure Instance Details](#)

Now, some network rules needs to be configured in order to expose some ports so the airflow server can be accessed:

Step 6: Configure Security Group

A security group is a set of firewall rules that control the traffic for your instance. On this page, you can add rules to allow specific traffic to reach your instance. For example, if you want to set up a web server and allow Internet traffic to reach your instance, add rules that allow unrestricted access to the HTTP and HTTPS ports. You can create a new security group or select from an existing one below. [Learn more](#) about Amazon EC2 security groups.

Assign a security group: ☒ Create a new security group ☐ Select an existing security group

Security group name:

Description:

Type	Protocol	Port Range	Source	Description
SSH	TCP	22	Custom 0.0.0.0/0	e.g. SSH for Admin Desktop
HTTP	TCP	80	Custom 0.0.0.0/0, ::/0	e.g. SSH for Admin Desktop
HTTPS	TCP	443	Custom 0.0.0.0/0, ::/0	e.g. SSH for Admin Desktop

[Add Rule](#)

Warning
Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

[Cancel](#) [Previous](#) [Review and Launch](#)

Now that it has been created, it can be accessed through an EC2 instance connection (just because its easier to configure intially):

Connect to your instance

Connection method: ☐ A standalone SSH client ☐ Session Manager ☒ EC2 Instance Connect (browser-based SSH connection)

Connect using a custom user name, or default to the user name for the AMI used to launch the instance. [Learn more](#)

User name:

[Close](#) [Connect](#)

Alarm Status: None
Public DNS (IPv4): ec2-35-180-33-242.eu-...
IPv4 Public IP: 35.180.33.242

When connected, the first thing to be done is to make sure every package is up to date:

```
eu-west-3.console.aws.amazon.com/ec2/v2/connect/ec2-user/i-07298310ca7c0c960

  _I_ ( _I_ / )
 _I_ ( _I_ / )   Amazon Linux 2 AMI
 _I_ \ _I_ \ _I_

https://aws.amazon.com/amazon-linux-2/
package(s) needed for security, out of 24 available
run "sudo yum update" to apply all updates.
ec2-user@ip-172-31-8-71 ~]$ sudo yum update -y
```

Then, install docker:

```
Complete!
[ec2-user@ip-172-31-8-71 ~]$ sudo amazon-linux-extras install docker
Installing docker
```

Starting the docker service:

```
[ec2-user@ip-172-31-8-71 ~]$ sudo service docker start
Redirecting to /bin/systemctl start docker.service
[ec2-user@ip-172-31-8-71 ~]$
```

After the docker service is started, just run a new docker using the rancher image (since it doesn't come installed within the machine, it has to be installed):

```
Last login: Fri Jan 24 14:08:56 2020 from ec2-35-180-112-82.eu-west-3.compute.amazonaws.com

  _I_ ( _I_ / )
 _I_ ( _I_ / )   Amazon Linux 2 AMI
 _I_ \ _I_ \ _I_

https://aws.amazon.com/amazon-linux-2/
[ec2-user@ip-172-31-8-71 ~]$ docker run -d --restart=unless-stopped --name rancher --hostname rancher -p 80:80 -p 443:443 rancher/rancher:v2.3.2
Unable to find image 'rancher/rancher:v2.3.2' locally
v2.3.2: Pulling from rancher/rancher
22e816666fd6: Extracting [=====] 17.69MB/26.69MB
079b6d2a1e53: Download complete
11048ebae908: Download complete
c58094023a2e: Download complete
8a37a3d9d32f: Download complete
e403b6985877: Download complete
9acf582a7992: Download complete
bed4e005ec0d: Download complete
74a2e9817745: Download complete
322f0c253a60: Download complete
883600f5c6cf: Download complete
ff331cbe510b: Download complete
e1d7887879ba: Download complete
5a5441e6019b: Waiting
```

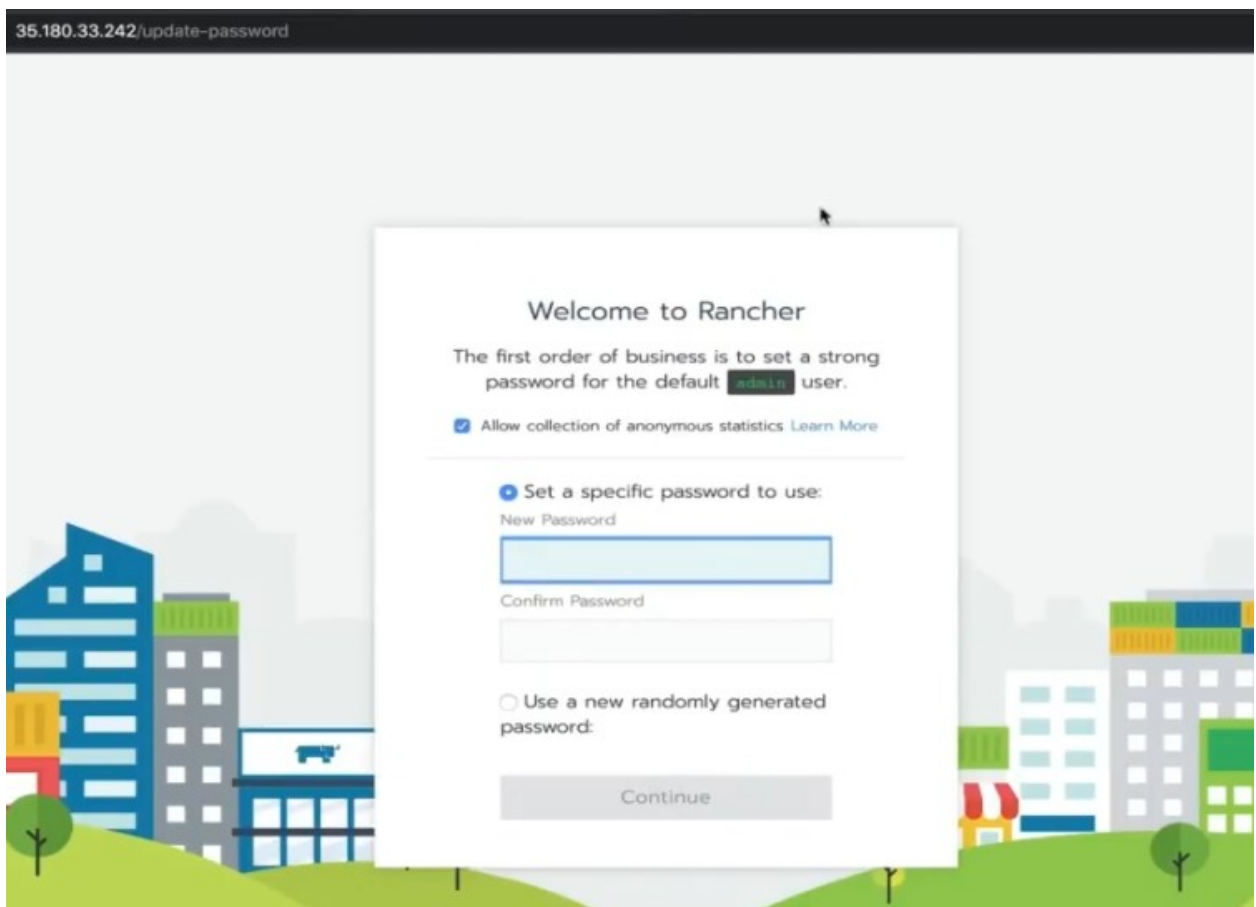
After it, just access the exposed IP:


```
MES
31c905f9ed13      rancher/rancher:v2.3.2  "entrypoint
ncher
[ec2-user@ip-172-31-8-71 ~]$
```

i-07298310ca7c0c960

Public IPs: 35.180.33.242 Private IPs: 172.31.8.71

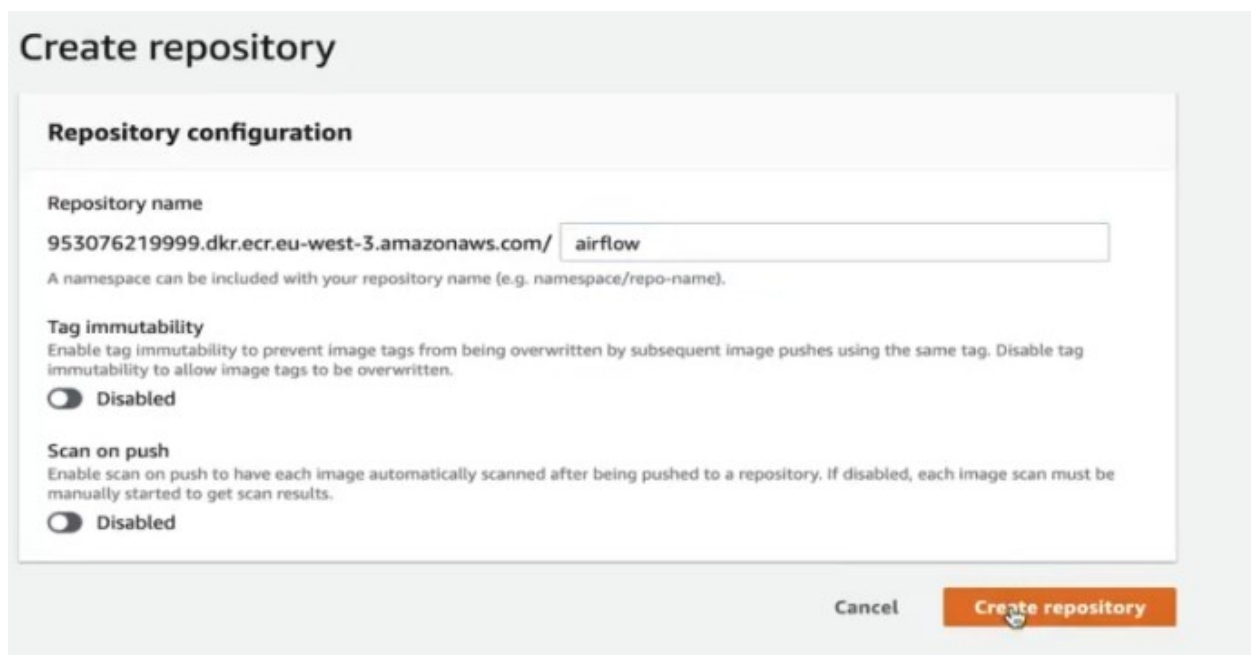
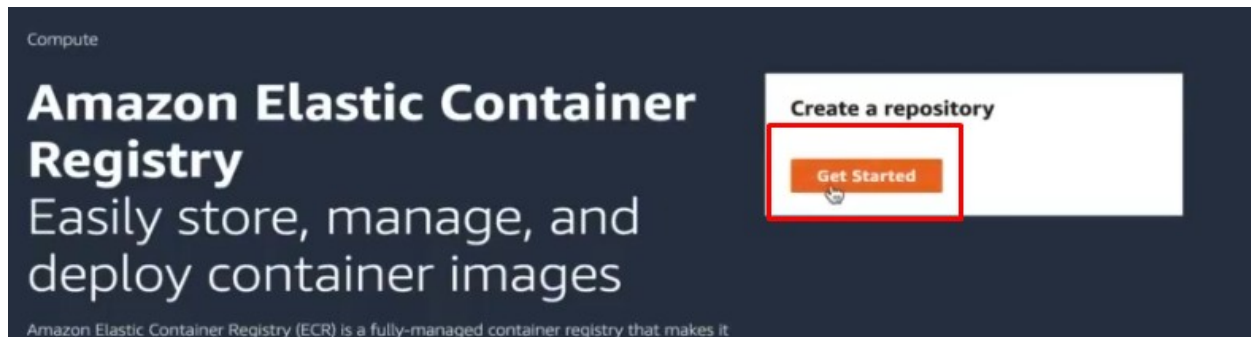
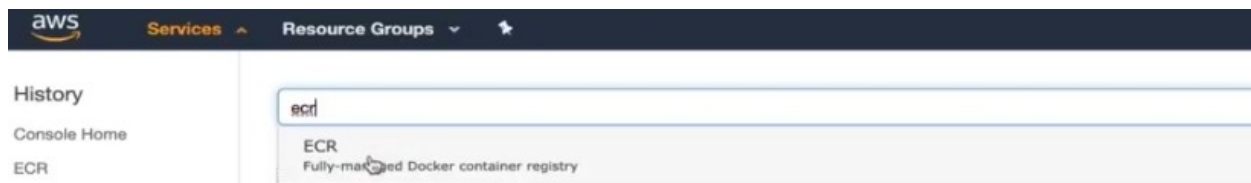
Now, Rancher should be successfully running in the AWS machine:



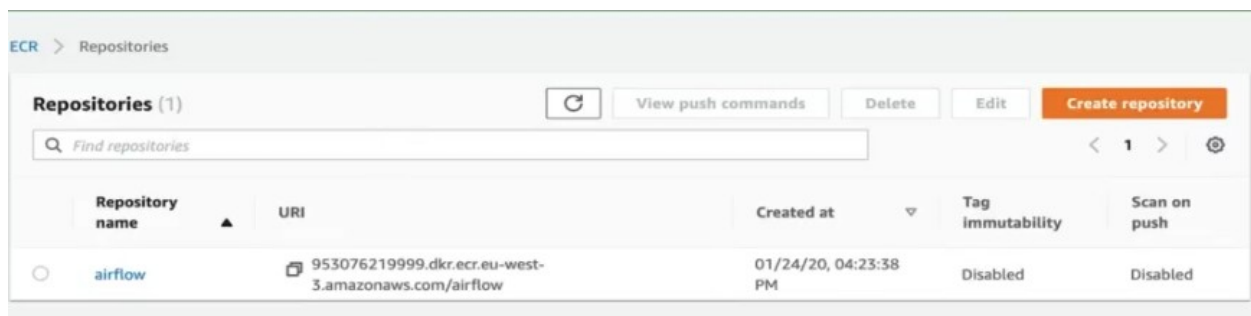
Creating an ECR repository

An ECR repository may be needed in order to store, manage and deploy docker containers.

From the AWS console, search for ECR and then click on get started:



Now, the repository should be created:



Now, docker images can be pushed into this repository, making it easier to use different custom images inside the AWS environment.

airflow View push commands

Images (1) Refresh Delete Scan

☐	Image tag	Image URI	Pushed at	Digest	Size (MB)	Scan status
☒	v1.0	953076219999.dkr.ecr.eu-west-3.amazonaws.com/airflow:v1.0	01/24/20, 04:45:15 PM	sha256:44f4d5935...	298.94	-

Creating an EKS Cluster with Rancher

Instead of importing a cluster, a new one will be created:

Rancher UI Navigation: Global, Clusters, Apps, Users, Settings, Security, Tools

Clusters Add Cluster

State Cluster Name Provider Nodes CPU RAM Search

There are no clusters defined.

Add Cluster - Select Cluster Type

From existing nodes (Custom)

Create a new Kubernetes cluster using RKE, out of existing bare-metal servers or virtual machines.

Import

Import managed cluster

With RKE and new nodes in an infrastructure provider

Amazon EC2

Azure

DigitalOcean

Linode

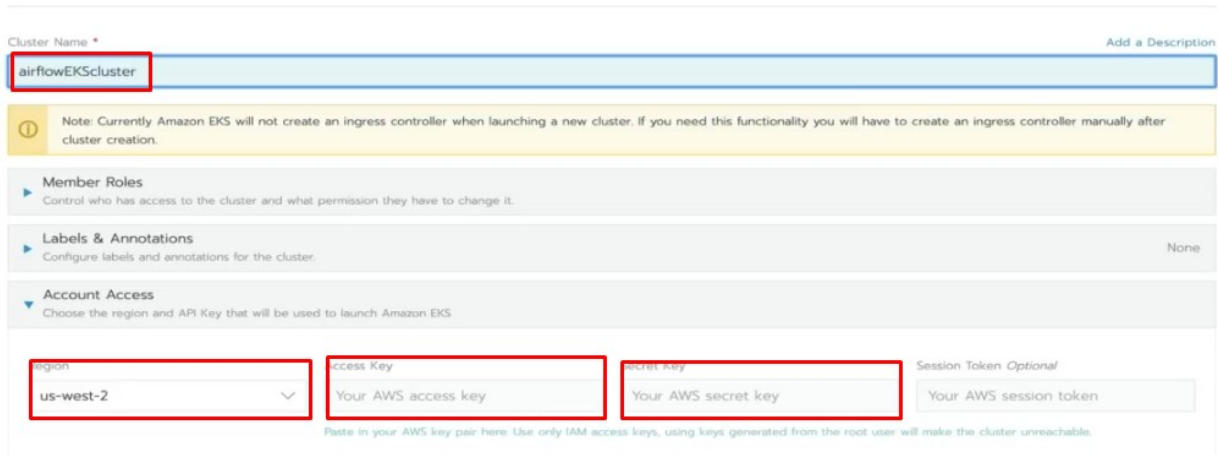
With a hosted Kubernetes provider

Amazon EKS

Azure AKS

Google GKE

Now, some AWS security credentials need to be filled up in order to continue configuring the cluster: region, access key and secret key, which were provided by AWS when its instance was initiated:



Cluster Name * Add a Description

airflowEKScuster

Note: Currently Amazon EKS will not create an ingress controller when launching a new cluster. If you need this functionality you will have to create an ingress controller manually after cluster creation.

Member Roles
Control who has access to the cluster and what permission they have to change it.

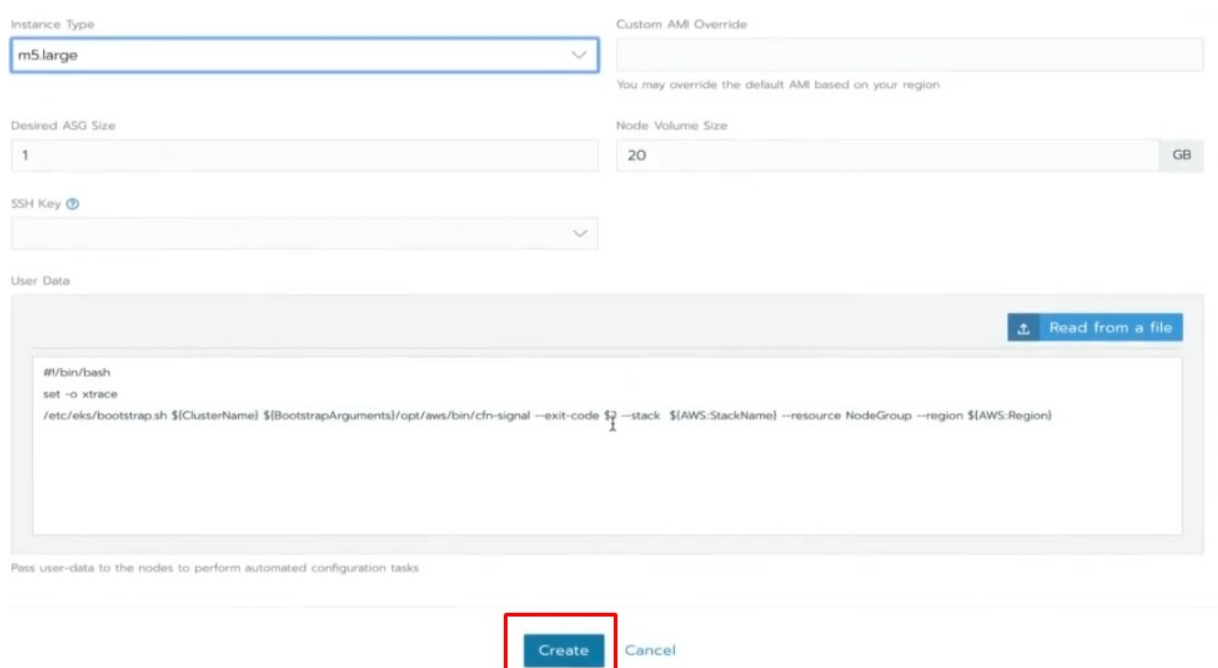
Labels & Annotations
Configure labels and annotations for the cluster. None

Account Access
Choose the region and API Key that will be used to launch Amazon EKS

region: **us-west-2** access Key: Your AWS access key secret key: Your AWS secret key Session Token Optional: Your AWS session token

Paste in your AWS key pair here. Use only IAM access keys, using keys generated from the root user will make the cluster unreachable.

The amount of nodes and other kinds of similar configuration can be configured before creating the cluster, but for the sake of simplicity, will be used the default settings. Click "Create":



Instance Type: **m5.large** Custom AMI Override

Desired ASG Size: **1** Node Volume Size: **20** GB

SSH Key ?

User Data

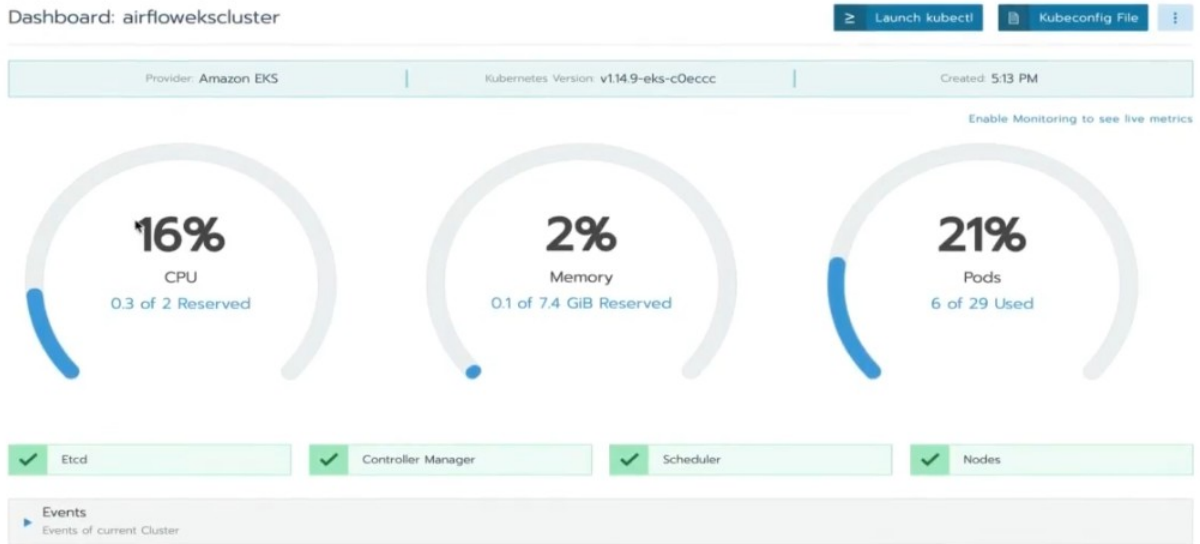
Read from a file

```
#!/bin/bash
set -o xtrace
/etc/eks/bootstrap.sh ${ClusterName} ${BootstrapArguments} /opt/aws/bin/cfn-signal --exit-code $? --stack ${AWS-StackName} --resource NodeGroup --region ${AWS-Region}
```

Pass user-data to the nodes to perform automated configuration tasks

Create Cancel

After the cluster is created, a dashboard can be accessed to visualize how its using the machine resources:



Launching a kubectl session

Dashboard: airflowekscluster

Launch kubectl | Kubeconfig File

Provider: Amazon EKS | Kubernetes Version: v1.14.9-eks-c0eccc | Created: 5:13 PM

Enable Monitoring to see live metrics

16% CPU
0.3 of 2 Reserved

2% Memory
0.1 of 7.4 GiB Reserved

21% Pods
6 of 29 Used

Shell: airflowekscluster | Connected

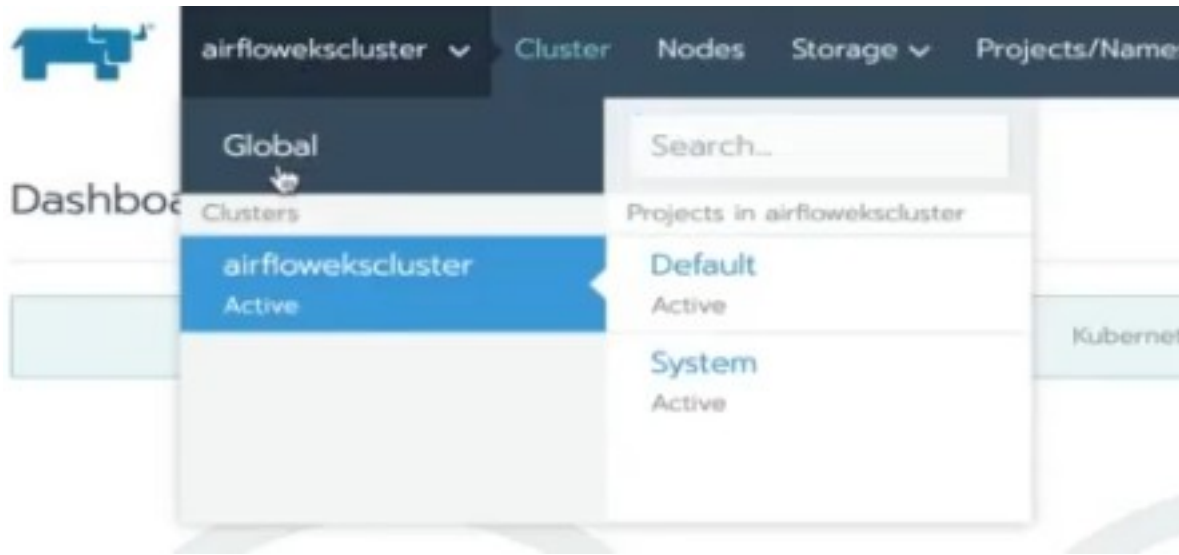
```
# Run kubectl commands inside here
# e.g. kubectl get all
>
```

Close

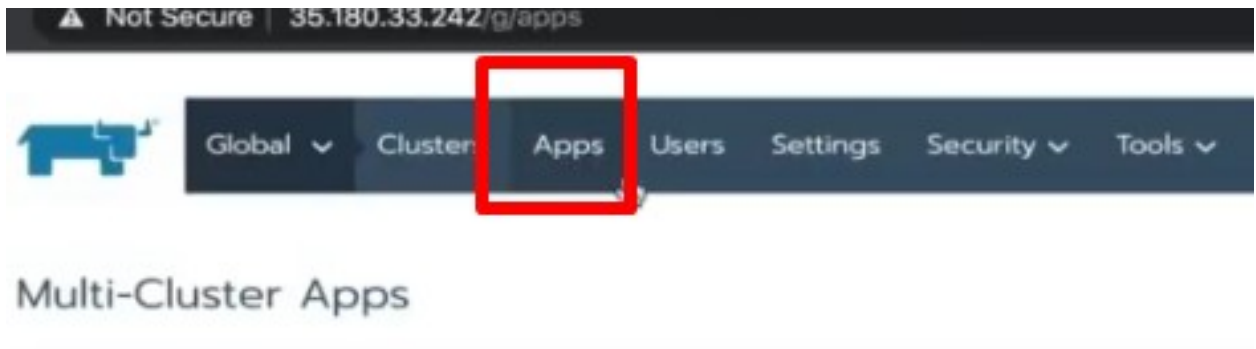
With this, it's possible to execute any kubectl command into the cluster. For example: `kubectl version` (for version), `kubectl get nodes` (list all the nodes being used by the cluster), etc.

Deploy Nginx Ingress with Catalogs (Helm)

Firstly, instead of selecting a single cluster, set it to global



Then, click on "Apps":



Then, "Manage Catalogs":



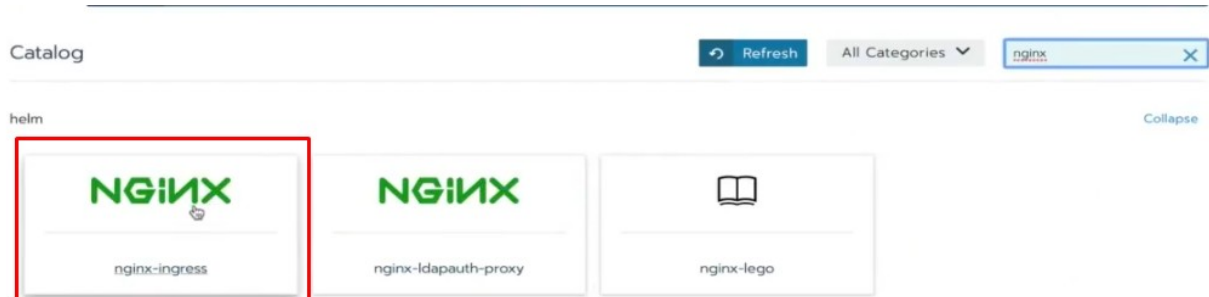
Now, enable the "helm" catalog:



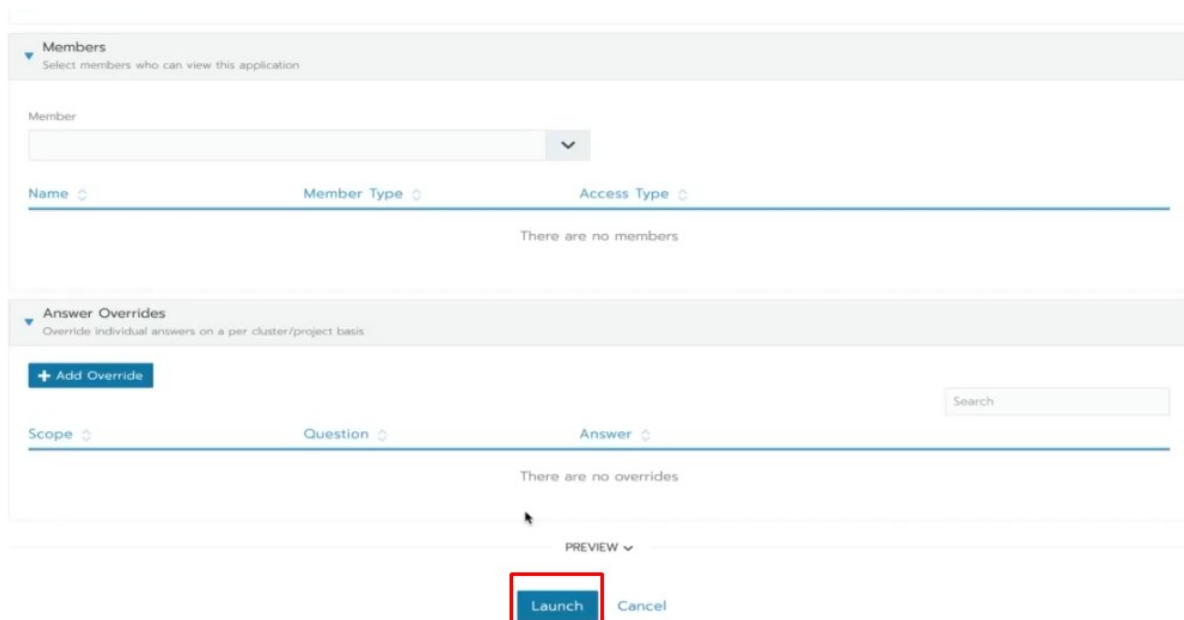
After it, it becomes active and starts to synchronize all the hand charts with Rancher. Once the sync is over, click on Apps again and then on Launch. With this, its possible to select many applications to be easily installed into rancher by using catalogs:



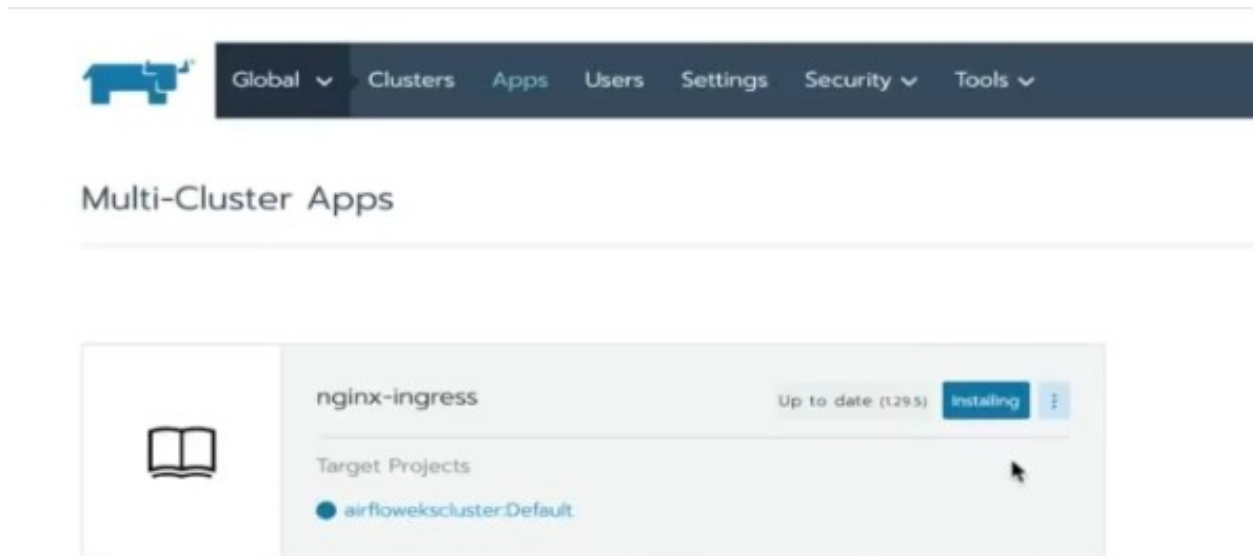
To install Nginx, just search for it then click on it:



After it, a page full of nginx configurations will pop up. After everything is configured nicely (will let the default settings, once again, for the sake of simplicity) and finally, click on "Launch":



Now, it should already appear as one of the Apps:

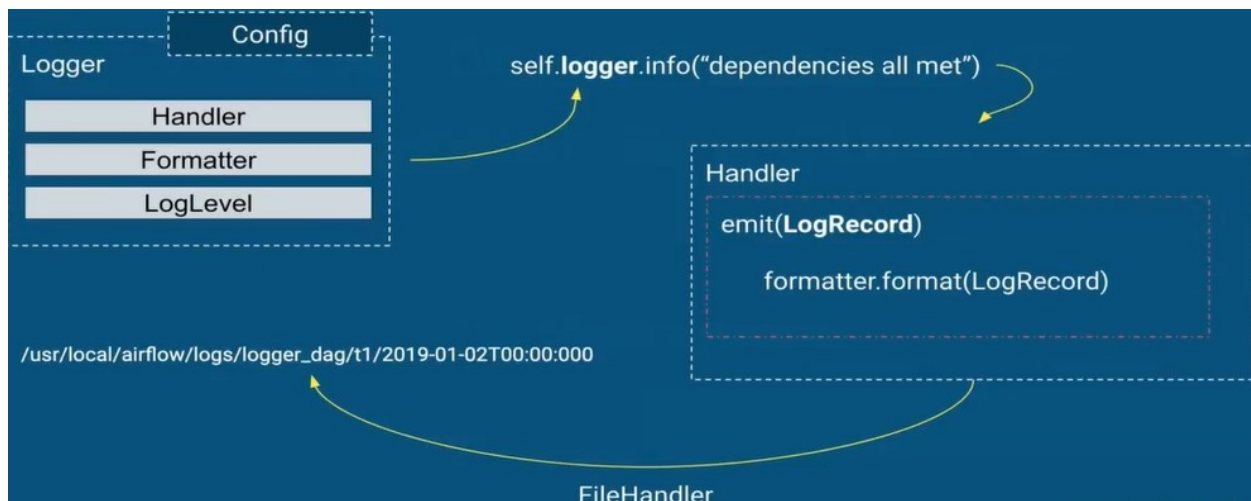


When clicked, will exhibit all the details related to its configuration and execution.

Monitoring Apache Airflow

How the logging system works in Airflow

- Based on the logging module
- Written into files
- Log levels (INFO, ERROR, DEBUG, WARNING, CRITICAL)
- Formatters
- Handlers for outputs:
 - FileHandler
 - StreamHandler
 - NullHandler



Setting up custom logging

In the airflow.cfg file, there are some important parameters to be set:

- `base_log_folder` is the parameter for setting the location where the logs will be stored.

```
# The folder where airflow should store its log files
# This path must be absolute
base_log_folder = /usr/local/airflow/logs
```

- `logging_level` is the parameter for setting which kind of information must be kept in the logs. By default, the value is set to `INFO`, meaning that any kind of information will be stored into the log file. If this is set to `WARNING`, only `WARNING` log levels will be stored into the log file.

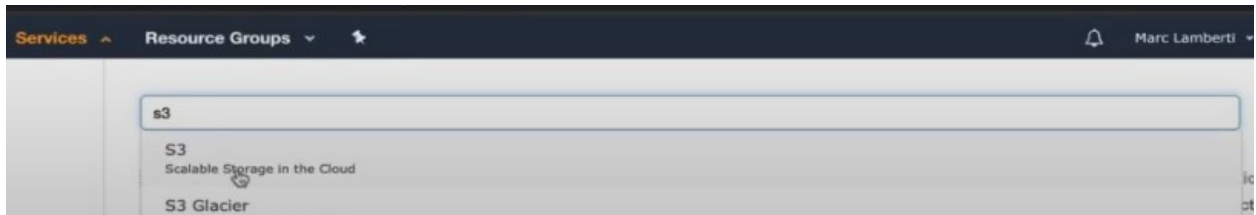
```
# Logging level
logging_level = INFO
fab_logging_level = WARN
```

- `log_format` defines the log format.

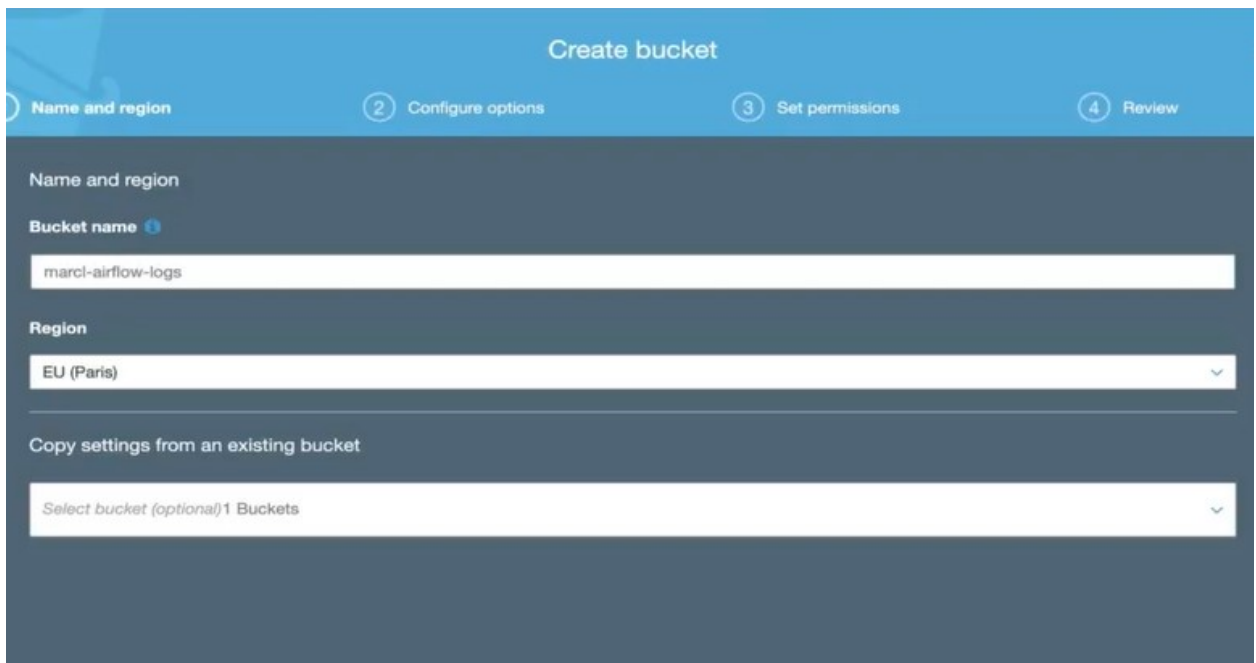
```
log_format = [%(asctime)s] {%(filename)s:%(lineno)d} %(levelname)s - %(message)s
simple_log_format = %(asctime)s %(levelname)s - %(message)s
```

Storing logs in AWS S3

It's also possible to store airflow logs on cloud services, like AWS S3. Firstly, search for S3 in the AWS dashboard:



Then, click on "create bucket":



Make sure its set to public access:

Note: You can grant access to specific users after you create the bucket.

Block public access (bucket settings)

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to all your S3 buckets and objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to your buckets or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

☒ **Block all public access**
Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

- ☐ **Block public access to buckets and objects granted through new access control lists (ACLs)**
S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.
- ☐ **Block public access to buckets and objects granted through any access control lists (ACLs)**
S3 will ignore all ACLs that grant public access to buckets and objects.
- ☐ **Block public access to buckets and objects granted through new public bucket or access point policies**
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.
- ☐ **Block public and cross-account access to buckets and objects through any public bucket or access point policies**
S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

[Previous](#) [Next](#)

Now, create the bucket:

Create bucket

1 Name and region 2 Configure options 3 Set permissions 4 Review

Name and region [Edit](#)

Bucket name marcel-airflow-logs **Region** EU (Paris)

Options [Edit](#)

Versioning	Disabled
Server access logging	Disabled
Tagging	0 Tags
Object-level logging	Disabled
Default encryption	None
CloudWatch request metrics	Disabled
Object lock	Disabled

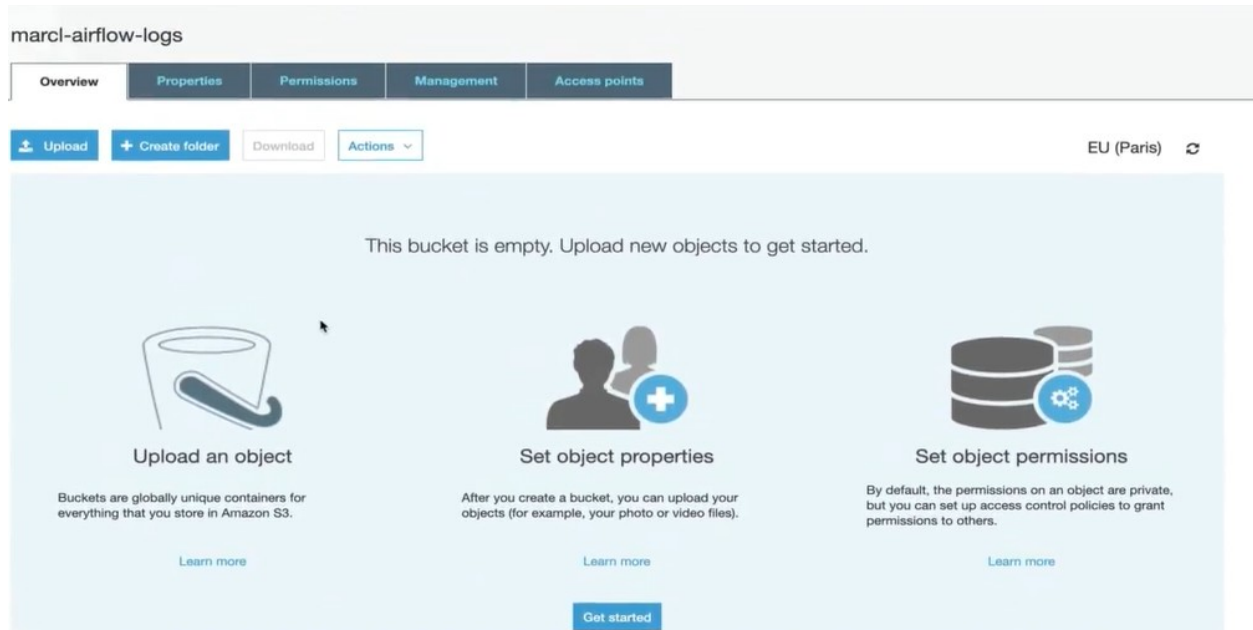
Permissions [Edit](#)

Block all public access
On

- Block public access to buckets and objects granted through new access control lists (ACLs)**
On

[Previous](#) [Create bucket](#)

Now, there should be an empty bucket ready to use



Setting up an user to access and modify data on this bucket

Here, a new user will be created, as well with its configuration for optimal access to the bucket.

Add user



Set user details

You can add multiple users at once with the same access type and permissions. [Learn more](#)

User name*

[+ Add another user](#)

Select AWS access type

Select how these users will access AWS. Access keys and autogenerated passwords are provided in the last step. [Learn more](#)

- Access type* ☒ **Programmatic access**
Enables an **access key ID** and **secret access key** for the AWS API, CLI, SDK, and other development tools.
- ☐ **AWS Management Console access**
Enables a **password** that allows users to sign-in to the AWS Management Console.

Add user

1 2 3 4 5

Set permissions

 Add user to group

 Copy permissions from existing user

 Attach existing policies directly

Create policy



Filter policies ▾ Showing 513 results

	Policy name ▾	Type	Used as
☐	 AdministratorAccess	Job function	Permissions policy (3)
☐	 AlexaForBusinessDeviceSetup	AWS managed	None
☐	 AlexaForBusinessFullAccess	AWS managed	None
☐	 AlexaForBusinessGatewayExecution	AWS managed	None
☐	 AlexaForBusinessPolyDelegatedAccessPolicy	AWS managed	None
☐	 AlexaForBusinessReadOnlyAccess	AWS managed	None
☐	 AmazonAPIGatewayAdministrator	AWS managed	None
☐	 AmazonAPIGatewayInvokeFullAccess	AWS managed	None

Set permissions boundary

Cancel Previous Next: Tags

Add user

1 2 3 4 5

Set permissions

 Add user to group

 Copy permissions from existing user

 Attach existing policies directly

Create policy



Filter policies ▾ Showing 513 results

	Policy name ▾	Type	Used as
☐	 AdministratorAccess	Job function	Permissions policy (3)
☐	 AlexaForBusinessDeviceSetup	AWS managed	None
☐	 AlexaForBusinessFullAccess	AWS managed	None
☐	 AlexaForBusinessGatewayExecution	AWS managed	None
☐	 AlexaForBusinessPolyDelegatedAccessPolicy	AWS managed	None
☐	 AlexaForBusinessReadOnlyAccess	AWS managed	None
☐	 AmazonAPIGatewayAdministrator	AWS managed	None
☐	 AmazonAPIGatewayInvokeFullAccess	AWS managed	None

Set permissions boundary

Cancel Previous Next: Tags

Now, a new policy will be created for this new user's usage:

Create policy

1 2

A policy defines the AWS permissions that you can assign to a user, group, or role. You can create and edit a policy in the visual editor and using JSON. [Learn more](#)

Visual editor JSON

[Import managed policy](#)

Expand all Collapse all

Select a service

Service

Choose a service

Actions

Choose a service before defining actions

Resources

Choose actions before applying resources

Request conditions

Choose actions before specifying conditions

[Add additional permissions](#)

For "Service", chose S3:

Visual editor JSON

[Import managed policy](#)

Expand all Collapse all

Select a service

Service

close

Select a service below

Actions

Choose a service before defining actions

Resources

Choose actions before applying resources

Request conditions

Choose actions before specifying conditions

Enter service manually

For "Actions", chose ListBucket, GetObject, PutObject, DeleteObject, ReplicateObject and RestoreObject.

List (1 selected)

HeadBucket

ListAllMyBuckets

☒ ListBucket

▼ ☐ Read (1 selected)

☐ DescribeJob ?	☐ GetBucketPolicyStatus ?	☐ GetObjectTagging ?
☐ GetAccelerateConfiguration ?	☐ GetBucketPublicAccessBlock ?	☐ GetObjectTorrent ?
☐ GetAccessPoint ?	☐ GetBucketRequestPayment ?	☐ GetObjectVersion ?
☐ GetAccessPointPolicy ?	☐ GetBucketTagging ?	☐ GetObjectVersionAcl ?
☐ GetAccessPointPolicyStatus ?	☐ GetBucketVersioning ?	☐ GetObjectVersionForReplication ?
☐ GetAccountPublicAccessBlock ?	☐ GetBucketWebsite ?	☐ GetObjectVersionTagging ?
☐ GetAnalyticsConfiguration ?	☐ GetEncryptionConfiguration ?	☐ GetObjectVersionTorrent ?
☐ GetBucketAcl ?	☐ GetInventoryConfiguration ?	☐ GetReplicationConfiguration ?
☐ GetBucketCORS ?	☐ GetLifecycleConfiguration ?	☐ ListAccessPoints ?
☐ GetBucketLocation ?	☐ GetMetricsConfiguration ?	☐ ListBucketMultipartUploads ?
☐ GetBucketLogging ?	☒ GetObject ?	☐ ListBucketVersions ?
☐ GetBucketNotification ?	☐ GetObjectAcl ?	☐ ListJobs ?
☐ GetBucketObjectLockConfiguration ?	☐ GetObjectLegalHold ?	☐ ListMultipartUploadParts ?
☐ GetBucketPolicy ?	☐ GetObjectRetention ?	

▼ ☐ Write (4 selected)

☐ AbortMultipartUpload ?	☐ PutBucketCORS ?	☒ PutObject ?
☐ CreateAccessPoint ?	☐ PutBucketLogging ?	☐ PutObjectLegalHold ?
☐ CreateBucket ?	☐ PutBucketNotification ?	☐ PutObjectRetention ?
☐ CreateJob ?	☐ PutBucketObjectLockConfiguration ?	☐ PutReplicationConfiguration ?
☐ DeleteAccessPoint ?	☐ PutBucketRequestPayment ?	☐ ReplicateDelete ?
☐ DeleteBucket ?	☐ PutBucketVersioning ?	☒ ReplicateObject ?
☐ DeleteBucketWebsite ?	☐ PutBucketWebsite ?	☒ RestoreObject ?
☒ DeleteObject ?	☐ PutEncryptionConfiguration ?	☐ UpdateJobPriority ?
☐ DeleteObjectVersion ?	☐ PutInventoryConfiguration ?	☐ UpdateJobStatus ?
☐ PutAccelerateConfiguration ?	☐ PutLifecycleConfiguration ?	
☐ PutAnalyticsConfiguration ?	☐ PutMetricsConfiguration ?	

Now, the created bucket must be selected in the Resources section:

▼ Resources ☒ Specific ☐ All resources [close](#)

bucket ?	Specify bucket resource ARN for the ListBucket action. Add ARN to restrict access	☐ Any
object ?	Specify object resource ARN for the PutObject and 4 more actions. ⓘ Add ARN to restrict access	☐ Any

Fill in with the desired bucket name:

Add ARN(s)✕

Amazon Resource Names (ARNs) uniquely identify AWS resources. Resources are unique to each service. [Learn more](#)

Specify ARN for bucket List ARNs manually

arn:aws:s3:::

Bucket name *

☐ Any

Cancel

Add

▼ Resources

close

☒ Specific

☐ All resources

bucket ?

arn:aws:s3:::marci-airflow-logs

EDIT

✕

☐ Any

Add ARN to restrict access

object ?

arn:aws:s3:::marci-airflow-logs/*

EDIT

✕

☐ Any

Add ARN to restrict access

Now, just set up a name for this policy and finish its creation:

Create policy

1

2

Review policy

Name* ReadWriteS3AirflowLogs

Use alphanumeric and '+@_.' characters. Maximum 128 characters.

Description

Maximum 1000 characters. Use alphanumeric and '+@_.' characters.

Summary

Filter

Service	Access level	Resource	Request condition
Allow (1 of 221 services) Show remaining 220			
S3	Limited: List, Read, Write	Multiple	None

* Required

Cancel

Previous

Create policy

Now that the policy has been successfully created, it should be attached to the new user:

Add user

1

2

3

4

5

Review

Review your choices. After you create the user, you can view and download the autogenerated password and access key.

User details

User name	airflow-log-s3
AWS access type	Programmatic access - with an access key
Permissions boundary	Permissions boundary is not set

Permissions summary

The following policies will be attached to the user shown above.

Type	Name
Managed policy	ReadWriteS3AirflowLogs

Tags

No tags were added.

Cancel

Previous

Create user

Now, this new user should now be able to add, modify and remove data from the bucket created.

Add user

1 2 3 4 5



Success

You successfully created the users shown below. You can view and download user security credentials. You can also email users instructions for signing in to the AWS Management Console. This is the last time these credentials will be available to download. However, you can create new credentials at any time.

Users with AWS Management Console access can sign-in at: <https://953076219999.signin.aws.amazon.com/console>

Download .csv

	User	Access key ID	Secret access key
▶	airflow-log-s3	AKIA53Z6FABPSTXVDOOT	***** Show

Completing the connection on airflow

Now that everything is set up, its only needed to complete the connection by creating a new connection on airflow:

The screenshot shows the Airflow Admin interface. The top navigation bar includes 'Airflow', 'DAGs', 'Data Profiling', 'Browse', 'Admin', 'Docs', and 'About'. The 'Admin' menu is open, showing options like 'Pools', 'Configuration', 'Users', 'Connections', 'Variables', and 'XComs'. The 'Configuration' option is highlighted with a red box. Below the menu, the 'DAGs' section is visible, showing a table of DAGs with columns for 'DAG', 'Schedule', 'Status', 'Last Run', and 'DAG Runs'. The table lists two DAGs: 'data_dag' and 'logger_dag'. The 'logger_dag' is currently 'On' and has a last run time of '2020-01-28 00:00'. The bottom of the screen shows pagination controls and a 'Hide Paused DAGs' link.

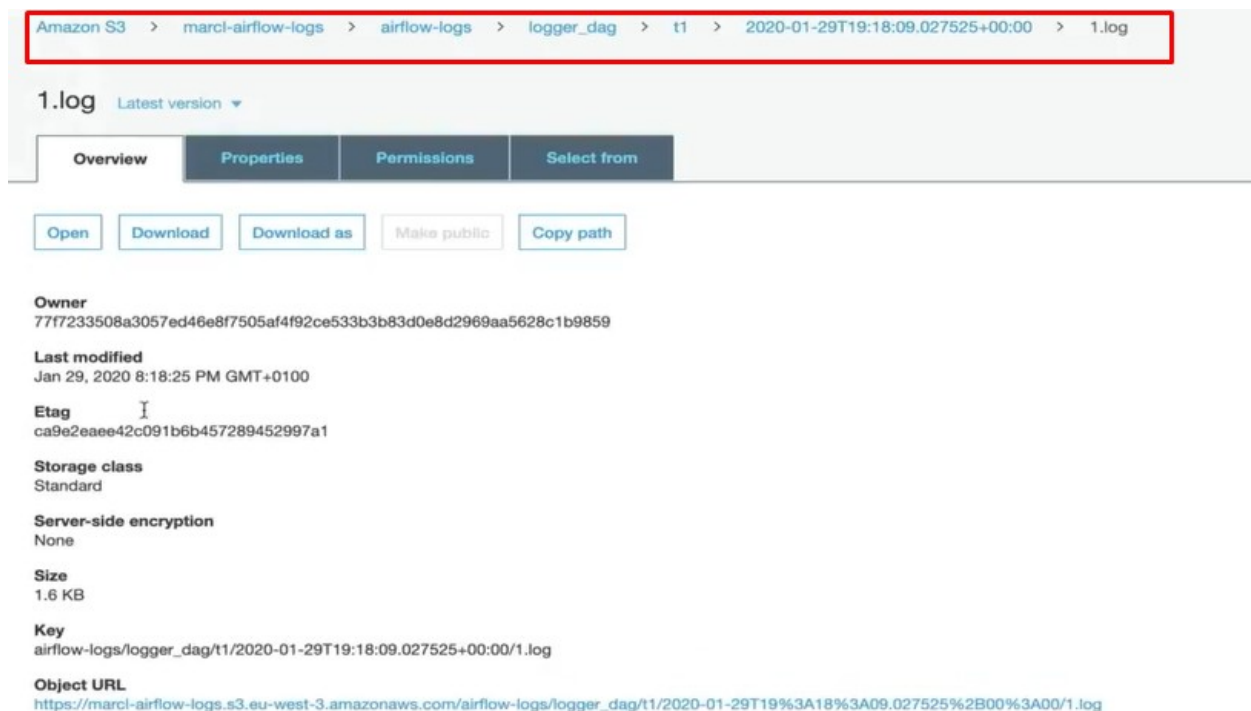
Now, set up the connection, specifying the AWS access key and secret access key:

The screenshot shows the 'Connection [create]' form in the Airflow Admin interface. The form has a 'List' tab and a 'Create' tab. The 'Create' tab is active. The form fields are: 'Conn Id' (AWSS3LogStorage), 'Conn Type' (S3), 'Host', 'Schema', 'Login', 'Password', 'Port', and 'Extra'. The 'Extra' field contains the JSON string: `{ "aws_access_key_id": "AKIA53Z6FABPSTXVDOOT", "aws_secret_access_key": "s6+L+Y6/q7+[1uxJb3nGLme6gUgVjP17wfCmPgSf" }`. At the bottom of the form, there are four buttons: 'Save', 'Save and Add Another', 'Save and Continue Editing', and 'Cancel'.

Now, in the airflow.cfg file, enable remote access, specify the name of the connection responsible for making remote access, as well as the directory in which the logs must be saved, which, in this case, will be the bucket created previously.

```
# Airflow can store logs remotely in AWS S3, Google Cloud Storage or Elastic Search.
# Users must supply an Airflow connection id that provides access to the storage
# location. If remote_logging is set to true, see UPDATING.md for additional
# configuration requirements.
remote_logging = True
remote_log_conn_id = AWS53LogStorage
remote_base_log_folder = s3://marcl-airflow-logs/airflow-logs
encrypt_s3_logs = False
```

Now, every log generated by airflow will be automatically saved into the AWS S3 bucket:



The screenshot shows the AWS S3 console interface. At the top, a breadcrumb trail is highlighted with a red box: Amazon S3 > marcl-airflow-logs > airflow-logs > logger_dag > t1 > 2020-01-29T19:18:09.027525+00:00 > 1.log. Below this, the file name '1.log' is displayed with a 'Latest version' dropdown. A tabbed interface shows 'Overview', 'Properties', 'Permissions', and 'Select from', with 'Overview' selected. Below the tabs are buttons for 'Open', 'Download', 'Download as', 'Make public', and 'Copy path'. The 'Overview' section displays the following details:

- Owner:** 77f7233508a3057ed46e8f7505af4f92ce533b3b83d0e8d2969aa5628c1b9859
- Last modified:** Jan 29, 2020 8:18:25 PM GMT+0100
- Etag:** ca9e2eaae42c091b6b457289452997a1
- Storage class:** Standard
- Server-side encryption:** None
- Size:** 1.6 KB
- Key:** airflow-logs/logger_dag/t1/2020-01-29T19:18:09.027525+00:00/1.log
- Object URL:** https://marcl-airflow-logs.s3.eu-west-3.amazonaws.com/airflow-logs/logger_dag/t1/2020-01-29T19%3A18%3A09.027525%2B00%3A00/1.log

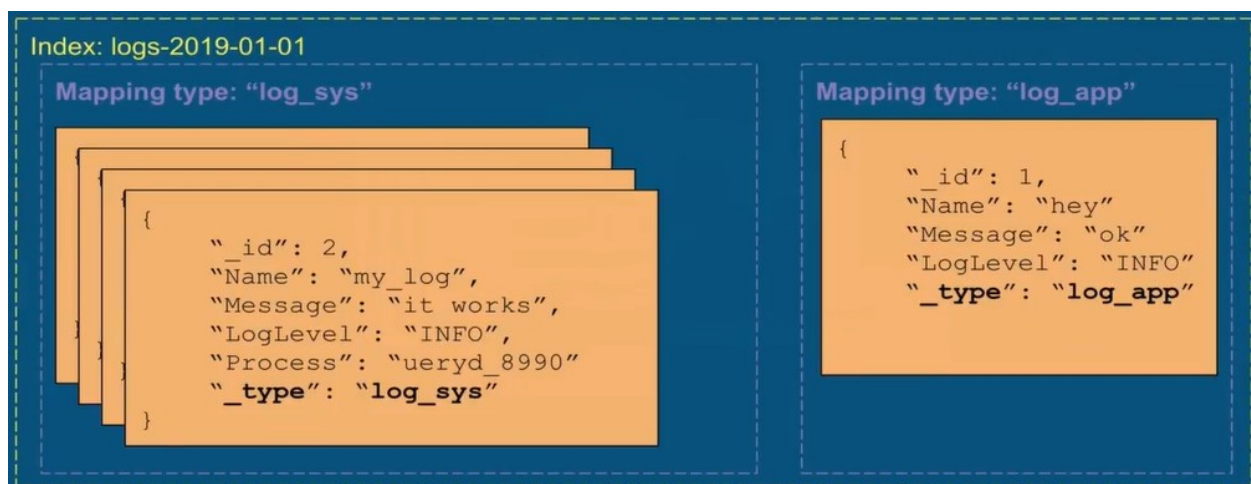
Elasticsearch

Elasticsearch is a document-oriented database that works based on two principal concepts:

- **Document:** a document can be seen as row in a relational database and it corresponds to your data stored in json format in elasticsearch, so basically a document could be the following json data coming from a log file with different fields such as id, name, and log level, as well with their associated values. Since your log file contains many log events, each log event is a document along with an unique ID, then these documents are stored into an index, which is a collection of documents.



Then, an index can have one or more mapping types that are used to divide documents into logical groups inside the same index. A mapping defines how a document is indexed and how its fields are indexed and stored. You can think of a mapping as a table in a relational database, basically, each document has a defined type as you can see from the field `_type`. Finally, once your documents are mapped and stored into an index, you can start requesting them through the rest API given by Elasticsearch.



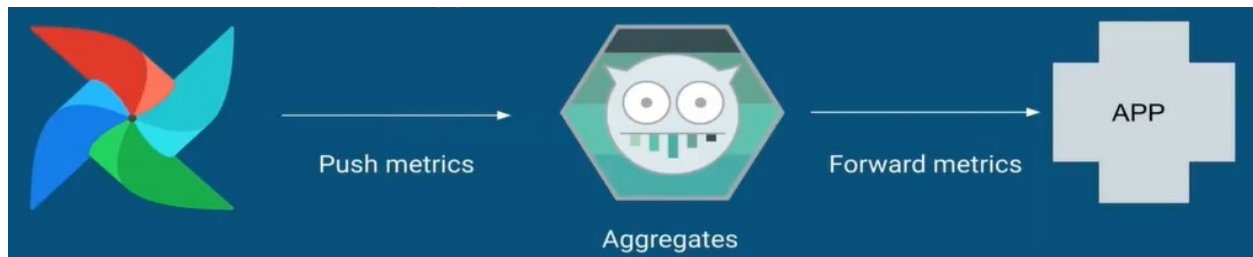
Introduction to metrics

Airflow updates different metrics such as the scheduler heartbeats to know if it's still up, the time taken to scan and import all DAG files at once, the number of running tasks on executor and so on. All of these metrics are sent to another tool called StatsD in order to export them.

So what is StatsD? StatsD is a simple daemon developed and released by hc in order to aggregate and summarize application metrics. You can think of StatsD as a push-based monitoring system where it receives logs from an application and pushes them to somewhere else such as Elasticsearch or InfluxDB, for example. Whenever Airflow wants to send a metric, this metric is sent over UDP datagrams to the statsd daemon.

But why UDP? Well, since this protocol doesn't wait for acknowledgement like with TCP, it's extremely fast and there is no risk of blocking the application waiting for StatsD to receive the metric.

- Airflow sends metrics to StatsD
- Daemon to aggregate and summarize application metrics
- Extremely fast (UDP) and tiny resource footprint
- Forward metrics to other applications

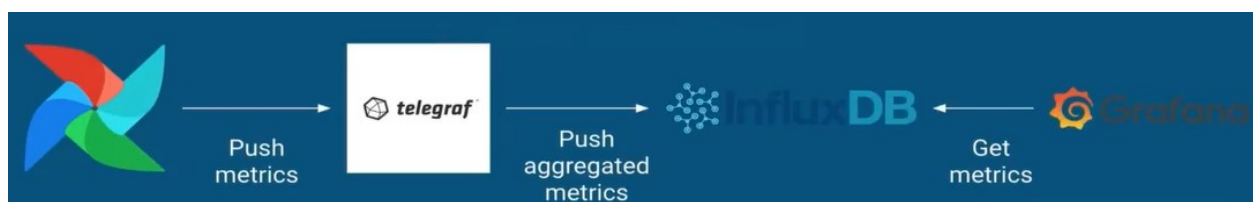


All metrics are based on three types:

- Counters (number of overall tasks failed,)
- Gauges (number of queued tasks on the executor)
- Timers (milliseconds taken to finish a task)

TIG Stack

- Telegraph
 - Agent for collecting, processing, aggregating metrics
- InfluxDB
 - Time series database
- Grafana
 - Data visualization and monitoring application



To connect Airflow to the TIG Stack, you can configure Airflow to export its metrics using the statsd or prometheus monitoring plugin. Telegraph acts as the data collector, receiving metrics from Airflow (via StatsD, Prometheus, or log parsing) and forwarding them to InfluxDB, a time-series database designed for storing and querying metrics. Grafana is then linked to InfluxDB as a data source, enabling visualization and monitoring of Airflow's performance and system metrics through customizable dashboards. This integration ensures seamless data flow from Airflow to Grafana via Telegraph and InfluxDB.

With this, it's possible to visualize how the data is behaving inside the airflow environment, making possible to better understand how the system is performing at different tasks.